

Causally aware reinforcement learning agents for autonomous cyber defence

Tom Purves^a, Konstantinos G. Kyriakopoulos^{b,*}, Siân Jenkins^a, Iain Phillips^c, Tim Dudman^a

^a Riskaware Ltd., Whitefriars, Lewins Mead, Bristol, BS12NT, UK

^b Wolfson School of Mechanical, Electrical and Manufacturing Engineering, Loughborough University, Loughborough, LE113TU, UK

^c Computer Science, Loughborough University, Loughborough, LE113TU, UK

ARTICLE INFO

Keywords:

Autonomous cyber defence
Reinforcement learning
Causal inference
Structural Causal Model

ABSTRACT

Artificial Intelligence (AI) is seen as a disruptive solution to the ever increasing security threats on network infrastructures. To automate the process of defending networked environments from such threats, approaches such as Reinforcement Learning (RL) have been used to train agents in cyber adversarial games. One primary challenge is how contextual information could be integrated into RL models to create agents which adapt their behaviour to adversarial posture. Two desirable characteristics identified for such models are that they should be interpretable and causal.

To address this challenge, we propose an approach through the integration of a causal rewards model with a modified Proximal Policy Optimisation (PPO) agent in Meta's MBRL-Lib framework. Our RL agents are trained and evaluated against a range of cyber-relevant scenarios in the Dstl YAWNING-TITAN (YT) environment. We have constructed and experimented with two types of reward functions to facilitate the agent's learning process. Evaluation metrics include, among others, games won by the defence agent (blue wins), episode length, healthy nodes and isolated nodes.

Results show that, over all scenarios, our causally aware agent achieves better performance than causally-blind state-of-the-art benchmarks in these scenarios for the above evaluation metrics. In particular, with our proposed High Value Target (HVT) rewards function, which aims not to disrupt HVT nodes, the number of isolated nodes is improved by 17% and 18% against the model-free and Neural Network (NN) model-based agents across all scenarios. More importantly, the overall performance improvement for the blue wins metric exceeded that of model-free and NN model-based agents by 40% and 17%, respectively, across all scenarios.

1. Introduction

Network security analysts struggle to recognise, prioritise, and promptly investigate collected evidence of attacks, due to the huge amount of collected data, false and missed alerts and lack of event semantics [1]. Amid the backdrop of a cyber-security arms race, policy makers have recognised AI solutions as a driver of both innovation and disruption within the cyber domain [2–5].

The behaviour of RL agents acting in an adversarial game is an interesting and challenging problem to analyse. At the core of this stands optimising rewards in a sequential decision making process, usually modelled by Markov Decision Process (MDP). Further challenges include data-inefficient setups [6], safety-critical environments [7,8], partially-observable [9] or ill-posed [10] systems, all of which raise unique concerns.

Aligning the operation of RL with human modes of decision making may provide inroads towards impactful technologies in regimes where

presently, humans alone are trusted to operate, such as in defence. One hallmark of cognition is that humans *model* their environments through experience. Driven by experimental successes [11–13], a rich literature in modelling using RL is evolving at a high pace. Two desirable characteristics identified for such models are that they should be interpretable and causal. In the context of causal inference, interpretability is the ability to understand, explain and intuitively capture and communicate the causal dynamics characterising a domain knowledge model [14].

State of the art methods for modelling agents interacting with an environment are often pursued in a RL paradigm. There are three challenges with current RL approaches. Firstly, in large scale deployments, the action space of RL agents is tremendously large, which is a problem if action/policy interplay is to be explored effectively [15].

Secondly, RL agents are generally modelled as a MDP. However, the Markov property states that the future state of a system depends only on the present state and action. This implies that the trajectory of all

* Corresponding author.

E-mail addresses: t.purves@hotmail.co.uk (T. Purves), elkk@lboro.ac.uk (K.G. Kyriakopoulos), sian.jenkins@riskaware.co.uk (S. Jenkins), i.w.phillips@lboro.ac.uk (I. Phillips), tim.dudman@riskaware.co.uk (T. Dudman).

<https://doi.org/10.1016/j.knosys.2024.112521>

Received 29 May 2024; Received in revised form 18 August 2024; Accepted 13 September 2024

Available online 17 September 2024

0950-7051/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

past events and actions is irrelevant to an agent's current state. This assumption is not valid in many real-world scenarios, where the state-action trajectory provides additional information that is necessary to consider for deploying the optimum action.

Finally, there is limited work within the intersection of Causal Inference (CI) and threat modelling domains to embed contextual information and provide situational awareness to RL defensive agents. CI [16] is a powerful method for determining actual cause-and-effect from indirect association. This, and other merits of causal models have been discussed at length by other authors [17,18].

The premise of this work is that domain knowledge can be leveraged as a priori knowledge to help towards modelling the cause-and-effect dynamics of the reward function, which would lead to guiding the RL agent in navigating the state and action spaces. At the intersection of RL and causal modelling, we produce causal models for networked scenarios and investigate how using these in adversarial settings may aid the training of RL agents towards acting optimally to defend a network from hostile or malicious parties.

With a causal model, it is possible, for instance, to estimate how the reward is affected based on the distance from an attacker to a critical asset. There is no such access to a refined cause-and-effect structure in a NN model. With a NN model, the best one could do is a black-box correlation study across inputs and outputs. With a graph like causal structure, the context (i.e. the other variables in the model) is automatically considered. Not only can one ask questions like 'What is the average reward obtained from reconnecting a critical asset?', but also 'What is the average reward obtained from reconnecting a critical asset in the cases where the attacker is nearby'. The intuition here is that if the attacker is close to a critical asset, one would rather try to first eliminate any surrounding attacker before reconnecting it.

This is the first work to utilise causal graphs that model the reward function of defence RL model-based agents trained in a cyber-security setting. As will be discussed in Section 4.5, the model-based reward was determined from a causal graph, rather than letting a NN emulate it. Taking such an approach replaces the black box nature of a NN with a human explainable and intuitive causal model and, furthermore, provides the capability for planning, which is omitted in model-free approaches.

The contribution of this article can be summarised as follows:

- Creation of novel causally aware RL agents that incorporate the cause-and-effect cyber dynamics, in order to model the rewards within a simulated cyber environment and optimise agent policy. Specifically, by leveraging expert knowledge on how cyber concepts impact each other, e.g. an "isolation" action increases the number of isolated nodes and relevant costs, we replace the NN reward model, improving the agent's policy.
- Comparison and evaluation of our proposed PPO causally aware model-based agent against two causally-blind agents: a model-free PPO agent and a NN model-based PPO agent. The hypothesis is that a causally aware version of the PPO agent would lead to improved behaviour.
- Evaluation of two different reward functions to identify additional benefits from agents that are encouraged to avoid isolating or restoring nodes that are of critical nature. In contrast to the standard rewards function, our proposed HVT rewards function lead to an improved health of the network, because less HVT nodes end up isolated by the end of the simulation game.
- Evaluation of the above approaches under a range of cyber scenarios, including low and high skilled adversaries and small and medium sized networks. The skill level concept is used to simulate different levels of threats, from low-skill attackers to Advanced Persistent Threat (APT) actors.
- To evaluate the performance we have used a plethora of evaluation metrics: win rate of agents, mean episode length, proportion of healthy and isolated nodes at the end of a simulated scenario, reward performance and fidelity, training and entropy loss.

This article is structured as follows. We discuss related work in Section 2 and cover the preliminary concepts on which we built upon in Section 3. In Section 4 we outline our approach to combining CI with RL for autonomous network defence. In Section 5 we analyse the results of training and evaluation in a range of cyber attack scenarios using two reward functions. In Section 6 we conclude, and in Section 7 we propose directions for future work.

2. Related work

Although a rapidly evolving research field, RL can be deployed today for solutions in natural language processing [19–21], robotics [22–24] and recommendation systems [25,26]. RL systems have also been identified as promising research approaches [27,28] for autonomous network defence agents. Model-based methods [29,30] have met success in game playing [13] as well as in optimal control tasks in robotics [31] and other real-world problems, where sampling the true environment is expensive.

There is also extensive literature on the application of model-based RL to cyber-security and cyber-defence [5,32]. Specifically, in terms of RL agents towards autonomous defence systems, the authors in [33] propose Double Deep Q-Networks (DDQN) [34] and Asynchronous Advantage Actor-Critic (A3C) [35] to train agents in selecting optimal actions, such as "isolate", "patch", "reconnect" and "migrate" network nodes. The ultimate goal of their approach was to protect HVT services. To this end, the reward function was informed by whether such services were maintained, shut down or migrated. The authors in [33] further investigate how adversarial RL could be used to address poisoning attacks against the defender's RL training process. Such attacks describe situations, where an attacker may alter reward signs or manipulate states to affect the learning process and lead to sub-optimal actions.

A limitation of such approaches is that they require training with perturbed data instances, in contrast to techniques that quantify the uncertainty of the decision making as an extra degree of insight for inferring that a perturbation attack is undergoing [36]. Such techniques do not require any training with perturbed datasets or any knowledge that an adversarial attack may take place. Furthermore, embedding causal inference in the learning process of the agent, which is the focus of our proposed work, is out of scope in [33].

In a similar spirit to [33] and without researching how to embed causal model dynamics in RL defensive agents (aka "blue" agents), the work in [27] proposes a hierarchical agent architecture, featuring a controller agent that can defend against two different types of APT adversaries. The controller is doing this by, firstly, identifying the type of adversary currently attacking the network and then selecting the most appropriate sub-agent to mitigate the specific adversary type. In other words, the hierarchical approach allows the defensive agent to predict the adversary within the early timesteps of an episode, and choose the most appropriate sub-agent to defend against it.

Each sub-agent is an RL agent itself trained, using an intrinsic reward, i.e. curiosity [37], to defend against a prescriptive and a more general APT adversary, respectively, as described in the Cyber Autonomy Gym for Experimentation (CAGE) Challenge [38]. The advantage of a hierarchical approach is that it provides a more generalised performance than the individual sub-agents, i.e. it has higher performance when the underlying adversarial type is unknown, which is usually the case. The work was implemented in the Cyber Operations Research Gym (CybORG) [39] environment.

The work in [28] is an extension of [27], focusing on the internal design of the sub-agents. The presented work demonstrates two new types of controllers, one heuristic and another bandit-based learning algorithm, to coordinate selection of which defensive sub-agent will deploy an action based on the sequence of observations they receive. Regarding the defence sub-agent NN structure, the authors have introduced two variations for the prescriptive APT adversary by

appending the observation with a single bit to indicate the success of the previously deployed action.

In contrast to the above works, our approach aims to extend the traditional Deep RL model-based approach by incorporating a causal representation of the factors contributing to the agent's reward. Our hypothesis is that using expert knowledge to model the reward dynamics will provide an advantage for the agent to navigate the cybersecurity RL environment, simulated in Dstl's YT [40]. Our work is inspired by the recent evidence that the Dynamic Causal Bayesian Optimisation (DCBO) algorithm [41], which is outside the remit of RL paradigms, permits optimal control of cyber-defence agents. DCBO is a dynamic extension to Causal Bayesian Optimisation (CBO), whereby the causal effects are evolving over time.

The authors in [42] have demonstrated how to provide optimal action recommendations to a BA within YT [40]. Three algorithms for decision making have been evaluated: Bayesian Optimisation (BO), CBO, DCBO. CBO and DCBO, due to their embedded causal relationships, converged faster in respect to the conventional BO. However, the temporal relationships did not offer any benefit towards convergence on the optimal solution. One of the assumptions by the authors is the absence of unmeasured confounders, i.e. variables that are associated with both the treatment and the outcome variables.

Regarding simulation environments, there are several options, such as YT [40], Primary-level AI Training Environment (PrimAITE) [43], CyBORG [39], CyGIL [44], FARLAND [45], CyberBattleSim [46] and NetSecGame [47]. We will focus on two environments: CyBORG and YT.

CyBORG is a Multi-Agent Reinforcement Learning (MARL) simulated environment for developing RL agents to address cyber security challenges. It supports a defender and an adversary agent, who interact with the turn-based environment using the actions available to them. Noteworthy, CyBORG also supports network emulation on top of Amazon Web Services (AWS) infrastructure, in order to bridge the performance divergence between simulations and reality. This is achieved by carefully designed overlapping actions among the various agents and their impact on the environment.

On the other hand, YT is an abstract, graph-based cyber-security simulation environment, where defensive autonomous agents face off against probabilistic red agents. It is therefore an ideal testbed for our prototype algorithm. YT follows the Gymnasium Application Programming Interface (API) [48], which permits communication between learning algorithms and environments. A wealth of literature is available on YT and its use in an applied cyber-security research testbed [28, 49–51].

3. Preliminaries

3.1. Reinforcement learning

RL [52] is one of the three major paradigms of machine learning, along with supervised and unsupervised machine learning. RL involves maximising the future rewards received by an agent from an environment, which is often modelled as a MDP [52]. Deep Reinforcement Learning (DRL) has come to be synonymous with the application of NN approaches to approximate the plethora of functions present in RL-solvable problems. The generalisability of neural networks as approximators for producing agents which are able to learn principles of optimal control without explicit reference to state action pairs makes them particularly suitable.

We take as our MDP the 4-tuple (S, A, P_a, R) , with S being a constant state space, A being an action space, $P_a(s, s') = \Pr(s'|s, a)$ the probability that action a in state s results in a new environment state s' , and R denoting the rewards issued to the agent from the environment under state transition from s to s' , when the action taken is a . The underlying RL paradigm of interest is a problem of maximising the expectation $E[\sum_{t=0}^{\infty} \gamma^t R]$ with discount factor $\gamma \in (0, 1)$ that reduces the

influence of future rewards and is governed by the Bellman equation for the value function with optimal policy V^* :

$$V^*(s) = \max_a \left(R + \gamma V(T(s, a)) \right) \quad (1)$$

where T is the associated transition operator for the MDP in question. While this traditional approach governs the *infinite horizon* MDP, similar methods are applicable to *finite horizon* environments, such as the cyber attack scenarios we construct.

Of particular interest to this work are *model-based* RL algorithms. The MDP modelling the environment is observed by a RL agent. This MDP, characterised by the transition function T and reward function R , is *learned* by the agent. The learned model then serves as a sample generator, permitting the agent to *imagine* experiences that *may never actually occur*. This is analogous to human cognition, where ‘the image of the world around us, which we carry in our head, is just a model’ [53], the RL agent is able to learn from experiences created in its own internal model.

3.2. Causal analysis

Cause and effect have recently been given a precise framework though the work of Pearl [54]. A symbiosis between nonparametric structural equations and a graphical approach is achieved through the *do-calculus*. Traditional statistical analysis¹ is geared to assess parameters of a distribution from samples drawn from that distribution. Using this, one can infer associations among variables, or the future likelihood of events based on past occurrence. Causal analysis goes beyond this regime as it enables a researcher to deliberately take an action to change the value of a variable of interest.² In other words, causal analysis investigates the impact of changing conditions, i.e. application of external interventions, to a system. Therefore, the phrase “*correlation does not imply causation*” is concretised into a useful principle.

Given that causal analysis is a proper and consistent science of intervention and observation, it seems natural that causal concepts would find ready application in model-based RL. Stated briefly, models of one's agency require a special kind of inference. Causal models make statements of the form *cum hoc ergo propter hoc* (‘with this, therefore because of this’).

The proposed approach suggests that since an agent may incorporate a model that reflects the environment (as the agent perceives it), such a model could be causal. Thus, an agent would be able to distinguish the dynamical system it is modelling, and its own place within that system. In addition, an interpretable modelling approach would enable practitioners to derive insights and communicate to peers or non-experts the underlying mechanisms that govern the model's dynamics. The latter would enable peers to further critique and validate the model [14]. To this end, a Structural Causal Model (SCM) is used as a model for the rewards R in our scenario and employ this in our model-based RL system. By explicitly modelling the causal dynamics among variables, SCMs make the underlying assumptions of causal relationships clear and transparent [16].

4. Methodology

4.1. Causal inference driven PPO algorithm

This section discusses the approach used to integrate model-based RL and causal modelling in the context of autonomous network defence. Specifically, our causally aware algorithm, titled CAICS, is inspired in part by the work in [13], and in part by DCBO [42].

Our proposed causally-aware agent design is based on the model-based approach depicted in Fig. 1, as proposed in the SimPLe algorithm [13]. In the SimPLe approach, there are three steps:

¹ Consider regression, estimation, and hypothesis testing techniques to illustrate what is meant by ‘statistical analysis’.

² This is known as an intervention or treatment in terms of causal analysis.

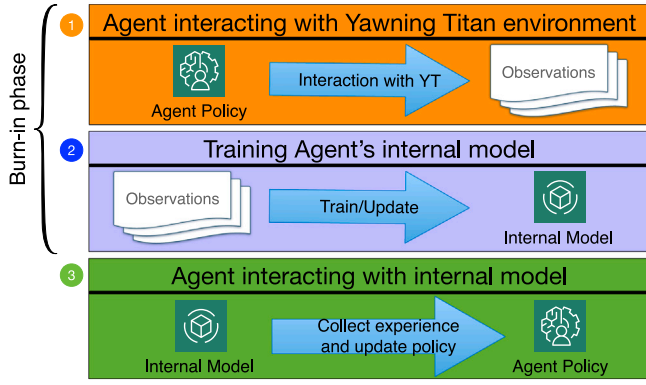


Fig. 1. An overview of the model-based approach with the three stages, as described in the SimPLE approach (adapted from [13]). The three stages are conceptually linked with the enumerated arrows in our proposed CAICS algorithm and workflow, as presented in Figs. 2 and 3, respectively.

1. The RL agent interacts with the YT environment following the most current policy (the policy is initialised randomly).
2. All obtained observations are used to train/update the agent's internal model.
3. The RL agent updates its policy by interacting with the internal model. The newly updated policy will be used to (i) evaluate the performance of the agent and (ii) collect more observations (i.e. back to step #1).

Fig. 2, depicts how the proposed, causally aware, agent collects experiences. As depicted, samples are generated from agent interactions with YT, the reference environment (left-hand side in Fig. 2) *env*, and the simulated environment (right-hand side) through the transition, T , and reward, R , models. Across each rollout,³ we use multiple, sim_{num} , modelled trajectories in tandem with a single reference environment trajectory. In our experiments, sim_{num} was set to 8 and the rollout horizon, n_{steps} , was set to 256.⁴

In “traditional” RL, the reward is obtained by playing a game in the environment. However, we may not have unlimited access to the environment and playing the game may take a “long” training time in terms of interaction steps with the environment. In contrast, the model-based advantage of sampling the simulated environment leads to less sampling of the reference environment during training at the expense of more computation, but it also allows the agent to learn from the introduced causally-aware reward model. Furthermore, an agent having access to a model enables it to estimate how a hypothetical action may affect the outcome, before the agent actually takes that action.

Recall that the essential ingredients for a model-based RL algorithm are some notion of a transition model $T : S \times A \rightarrow S$, which is a probabilistic mapping from one state to the next, and a rewards model $R : S \times A \rightarrow \mathbb{R}$. Together, (T, R) , allow the agent to simulate the true MDP environment, *env*.

In [13], the rewards model is learned with a NN, along with the transition model. We make use of a rollout buffer, aka as a State–Action–Reward–State–Action (SARSA) buffer, *buf*, in order to store these transitions. However, instead of relying on a NN to fit our rewards model R , our approach is to model the rewards model R with an

³ In RL, a “rollout” is the process of running a policy in the environment to generate a trajectory of states, actions, rewards, and next states.

⁴ The rollout horizon is a hyperparameter that determines the length of the rollout or the number of steps that the agent takes during each simulation. The length of the rollout horizon can have a significant impact on the performance and convergence of the reinforcement learning algorithm.

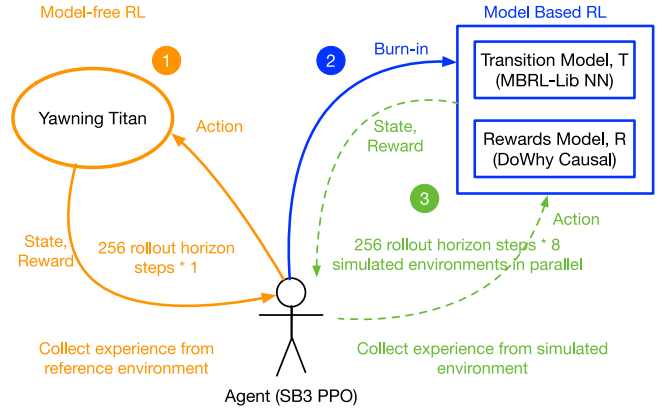


Fig. 2. Our algorithmic approach based on the Dyna framework [55] and the SimPLE approach [13]. The numbers link the algorithmic approach to the SimPLE model-based approach (Fig. 1) and the proposed workflow (Fig. 3). Models of the 8 simulated environments are fitted to data collected during beginning of the simulation, which we refer as the “burn-in” phase. Each model (T, R) is populated via the variables shown under burn-in phase, in Fig. 3.

SCM. Fig. 3 shows the workflow for populating the various buffers and models, their relation to each other and to the environment.

We outline the on-policy version of our algorithm in Algorithm 1. The policy and value functions of the PPO agent PPO_{π} are updated (line 12) making use of a Stable Baselines 3 (SB3) PPO buffer, PPO_{buf} , and the samples from the reference and simulated environments, as shown on lines 6 and 9, respectively. The agent in Algorithm 1 is fundamentally agnostic to the source of data samples. The size of the PPO_{buf} is a hyperparameter for the problem.

We also experimented with an off-policy approach, using a Soft Actor Critic (SAC) agent. The off-policy case is a simple alteration, where PPO_{π} is replaced with SAC_{π} , and the PPO_{buf} is combined with the SARSA buffer *buf*. This off-policy algorithm is closely analogous to the Model-Based Policy Optimisation (MBPO) algorithm [56], included in the MBRL-Lib framework. However, even though the SAC agent can handle both discrete and continuous observation spaces, its use is more appropriate for continuous action spaces. Thus, because the action space of the chosen environment (see Section 4.2) is discrete, the SAC agent underperformed in comparison to the PPO agent and, therefore, we do not include any off-policy results in this article. PPO natively accepts discrete/multidiscrete action spaces, congenial for RL in discrete environments, whereby SAC does not.

Algorithm 1 On-policy model based RL with causal inference

Require: $PPO_{\pi}, PPO_{buf}, env, buf, sim_{num}$

```

1: while init do
2:    $buf \leftarrow env$  ▷ Burn-in phase.
3: end while
4:  $T, R \leftarrow buf$  ▷ Initial fitting.
5: while not done do
6:    $PPO_{buf} \leftarrow PPO_{buf} \cup GET(env, PPO_{\pi})$ 
7:   ▷ Get experience from reference env.
8:   for  $i \in sim_{num}$  do
9:      $PPO_{buf} \leftarrow PPO_{buf} \cup GET((T, R), PPO_{\pi})$ 
10:    ▷ Get simulated experience.
11:   end for
12:    $PPO_{\pi} \leftarrow TRAIN(PPO_{buf})$  ▷ Train RL.
13: end while

```

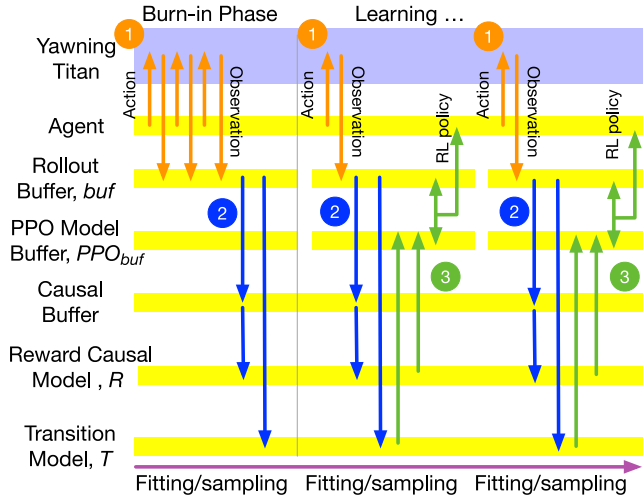


Fig. 3. Workflow showing how the algorithmic variables are populated. The numbers link the proposed workflow to the algorithmic approach (Fig. 2) and the SimPLE model-based approach (Fig. 1). The agent takes actions in the reference environment (*env*). The results of this experience are stored in the rollout buffer, *buf* (#1), and used to fit a reward model (#2) which, in turn, is used to generate additional training samples (#3). The RL policy is updated with the rewards obtained from both the reference (YT) and simulated (modelled) environments (#3), as seen in lines 6, 9 and 12 in Algorithm 1.

4.2. Simulated cyber security environment

In order for our agent to interact with a cyber-security relevant environment, we turn to YT [40]. YT is highly configurable, and a particular configuration will hereafter be referred to as a *scenario*. In YT there are two agents present; these are an adversary (red) agent, with configured skill level *RS* forming part of the scenario (as described in Section 4.4) and a defending (blue) agent.

A graph comprised of individual nodes *n* represents networked devices, which are either compromised or not compromised. The nodes have a randomly assigned vulnerability score *vuln*(*n*). Connections between devices are represented as edges in the graph. The scenario is turn-based, and the order of execution is as follows:

1. The red agent performs a turn.
2. The simulator checks if the red agent has won.
3. The blue agent performs a turn.
4. The environment rewards the blue agent.

The red turn is probabilistic, meaning the actions of the red agent are modelled without a particular RL policy or strategy. At each turn, the red agent attacks from a compromised node to an uncompromised node⁵ which, if successful, it immediately moves to and compromises. Attacks are deemed successful when the attack score exceeds a randomly generated integer between 0 and 100, as described by:

$$RS^2 / (RS + (1 - vuln(n))) \geq u \sim \mathcal{U}(0, 1) \quad (2)$$

The attack score provides a probabilistic model of real-world attack scenarios, where the probability of success is determined not only by the red agent's skill (*RS* as seen in the denominator of Eq. (2)), but also by the target node's defences ($1 - vuln(n)$). The numerator is the square of the red agent's skill level, in order to emphasise its impact. By squaring the skill level and taking the defence into consideration, the formula simulates a non-linear interaction between these factors, reflecting the complexity of cyber defence mechanisms [40].

⁵ In this scenario, the attack surface [57] is the set of nodes which are less than a single hop in the graph from any currently compromised node.

If uninterrupted by the blue agent, the red agent compromise spreads through the network. There also exist nodes marked as 'entry points' to the network, which can be attacked by the red agent at any point in the simulation, irrespective of any compromised status.

At each blue turn, the blue agent receives an observation of state *s* from the environment, and decides on an action *a*. The action *a* is picked from the set {*ReconN*, *IsoN*, *RestN*, *ReduV*}, which respectively denotes the reconnect node (which reconnects a device to the network, emulating removing blanket firewalls or power-up), isolate node (which simulates applying blanket firewalls or power-down), restore node (which simulates a reset of the device to a backup state), and reduce vulnerability (which simulates enhancement of security controls on the device, such as updating old software to patch vulnerabilities) actions respectively.

Encoded within state *s* are quantities concerning the health of the network, flags signifying compromise, the current adjacency matrix for the network, flags denoting particularly important devices on that network (e.g. the HVT and the entry points to the network), as well as the node vulnerability score.

The game is won by the red agent if the compromise spreads to a HVT. We define the blue agent as the winner when the simulation length exceeds an upper bound on steps, arbitrarily set here to be 500.

4.3. Reward functions

We have constructed and experimented with two types of reward functions to facilitate the learning process. A standard reward function and a modified one that provides the blue agent with additional context about the type of node on which it deployed an action. i.e. whether the node is a HVT or not.

4.3.1. Standard reward function

The standard reward function is the weighted sum of the number of nodes compromised/isolated and the *change* in number of nodes compromised/isolated. The former, represents the health of the network and the latter indicates whether the last action deteriorated or improved the health of the network. In other words, the standard reward function for step *t* is defined as:

$$R = \sum_{n=0}^N \left(\alpha_{iso} I_t^{iso}(n) + \beta_{iso} (I_t^{iso}(n) - I_{t-1}^{iso}(n)) + \alpha_{comp} I_t^{comp}(n) + \beta_{comp} (I_t^{comp}(n) - I_{t-1}^{comp}(n)) \right) + C_t + R_t^{TERM}, \quad (3)$$

where $I_t^{iso}(n)$ is the indicator function over the isolation status of nodes (I_t^{iso} returns 1 iff evaluated on an isolated node and a 0 otherwise) at time *t*, $I_t^{comp}(n)$ is the indicator function over compromised status of nodes at time *t*, with (*t* - 1) terms representing indicator functions for compromise and isolation status at time step *t* - 1. Therefore, $\sum_n I_t^{iso}(n)$ represents the number of isolated network nodes at time *t*, and similarly for compromise. The term

$$C_t = C_t^{ReconN} + C_t^{IsoN} + C_t^{RestN} + C_t^{ReduV}$$

represents an inherent cost applied for performing one of the four actions. R_t^{TERM} represents the rewards issued for blue winning or losing the game, fixed at +1000 and -1000 respectively. The coefficients α_{iso} , α_{comp} represent costs for nodes that are already isolated or compromised (i.e costs associated with the current health of the network), respectively. Similarly, the coefficients β_{iso} , β_{comp} represent costs for nodes *becoming* isolated or compromised after an action is taken,⁶ respectively.

⁶ Note that this number is distinct from the raw cost associated with the *IsoN* action. It is possible to isolate a different number of nodes by application

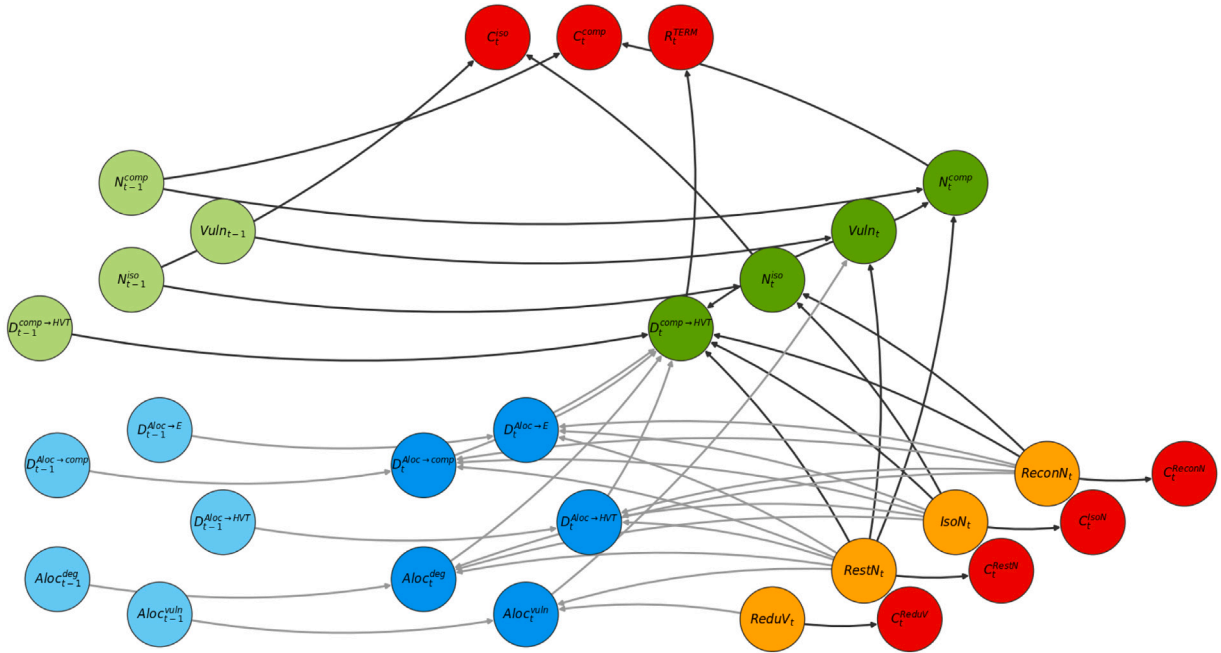


Fig. 4. The SCM used to model rewards in all scenarios. Grey lines and black lines have no special significance other than to aid visual clarity. Blue nodes represent variables concerned with action-locations. The green nodes are variables that consider network state. The orange nodes represent variables which can have interventions performed upon them. The red nodes represent costs and are summed to give total rewards. Lighter colours indicate variables that represent network dynamics on their darker coloured twins. The target variable is the Total reward at time t , R_t^{tot} (not shown above), which is composed of all the relevant costs.

We set $\alpha_{iso} = 0.025$, $\alpha_{comp} = 0.025$, $\beta_{iso} = 1$, $\beta_{comp} = 2$, which reflects the physical intuition that becoming compromised is twice as 'bad' as becoming isolated. But once isolated/compromised, a node deviating from the ideal operating circumstance in any way results in equally harsh penalties issued to the agent. These coefficients are simplistic assumptions, but provide a suitable level of richness to the rewards that allows the construction of a representative causal model for our research.

The α and β parameters, along with the action costs are defined within the `reward_functions.py` file in YT:

```
# Raw cost for performing actions
action_cost = {
    "reduce_vulnerability": 0,
    "restore_node": 1,
    "isolate": 1,
    "connect": 0,

# Costs associated with leaving nodes compromised or isolated
state_based_costs = {
    "per_node_compromised_cost": 0.025,
    "per_node_isolated_cost": 0.025

# Costs associated with a change in state
transition_based_costs = {
    "node_connection_change_cost": 1,
    "node_compromised_change_cost": 2
}
```

of *IsoN* action. The number of nodes that actually get isolated depends upon network topology. A similar illustration works for compromised nodes, application of *RestN* to an uncompromised node will not result in a change in the compromised status of the network but does still cost the agent to implement.

4.3.2. HVT reward function

The intuition behind designing this reward function is that we would like to train agents that avoid isolating or restoring HVT nodes, since, in real applications, tampering with an HVT node would have a higher Confidentiality, Integrity, and Availability (CIA) impact on the mission function provided by the network.

Thus, if the action is applied on an HVT node, the respective restoration and isolation costs are *doubled* in comparison to the costs of the standard reward function. Notably, this cost alteration for the HVT reward function did not require any modification to the SCM because all relevant concepts, such as the distance between the location of the node where the action was applied and the HVT, were already included in the SCM (as will be detailed in Section 4.5). The following code was part of the HVT reward function implementation:

```
important_nodes = network_interface.get_high_value_nodes()
cost = action_cost[blue_action]
if blue_node in important_nodes:
    cost *= 2
```

4.4. Cyber security scenarios

Four distinct scenarios have been constructed for evaluation of our agent. These are:

- Low skill adversary, small network
- High skill adversary, small network
- Low skill adversary, medium network
- High skill adversary, medium network

The red skill, *RS*, concept is used to model different levels of threats, which helps in simulating a variety of scenarios in cybersecurity training and testing within YT. The higher the *RS* level the more likely for the attack to succeed [40]. The *RS* levels are set in the `game_modes` configuration file within the YT file structure, with the low skill adversary acting with both a low skill level, $RS = 0.33$, and selecting a random node (from all possible targets) as a target at

each turn. The high skill level adversary has a higher configured skill level, $RS = 0.67$, and selects the most vulnerable node as a target at each turn. These configurations attempt to simulate different levels of threats, from low-skill attackers to APTs.

The number of nodes is arbitrarily set to 18 and 25 for the small and medium networks, respectively, based on the provided YT examples [40], and aiming to compare two network sizes, one nearly 50% larger from the other. The focus is on the algorithmic performance difference between these network sizes, as will be discussed in Section 5, rather than on the absolute number of nodes.

4.5. Causal model design

In this section we discuss the causal model we have constructed to describe the connections between Eq. (3) and state as observed (and modelled internally) by our agent. The Directed Acyclic Graph (DAG) we consider is depicted in Fig. 4. We note that our SCM is dynamical in nature, i.e. the SCM depends on variables *just before* and *just after* an intervention takes place.

This model serves to represent dependencies between variables that can be measured in state s . The agent is able to use SARSA type [52] transition buffers to fit this causal model on the data that it obtains during training (as depicted in Fig. 3).

The arrows in the causal model are modelled with regressors, random-forests or empirical distribution fit provided by scikit-learn [58]. These three approaches were used to train the causal model with the respective dependencies of the fit, i.e. the independent variables used in the algorithm, given by the connections in the DAG shown in Fig. 4. Root nodes in the DAG are characterised by empirical distributions in the SCM. Linear regressions were chosen when the underlying function to be learned is indeed linear. Random forests cover the complicated dependencies and are chosen for their excellent speed and performance record.

The causal graph was designed by identifying concepts within the YT environment that would contribute to the reward obtained after the blue agent deploys an action. The causal graph essentially captures how the reward value would behave by considering which action is deployed, on which network node this action is applied and other situational awareness related concepts. Specifically, the causal graph is composed of the following concepts:

Situational awareness concepts about network status (green nodes) fit empirically:

N_{t-1}^{comp}	Number of nodes compromised at $t - 1$
$Vuln_{t-1}$	Vulnerability in network at $t - 1$
N_{t-1}^{iso}	Number of nodes isolated at $t - 1$
$D_{t-1}^{comp \rightarrow HVT}$	Red to HVT distance at $t - 1$

Concepts related to action location (blue nodes) fit empirically:

$D_{t-1}^{Aloc \rightarrow HVT}$	Distance from action location to HVT at $t - 1$
$D_{t-1}^{Aloc \rightarrow E}$	Distance from action location to entry point at $t - 1$
$D_{t-1}^{Aloc \rightarrow comp}$	Distance from action location to Red Agent at $t - 1$
$Aloc_{t-1}^{deg}$	Action location degree at $t - 1$
$Aloc_{t-1}^{vuln}$	Action location vulnerability at $t - 1$

The same concepts as above are repeated for time instance t and fit with random forest except for $Vuln_t$ and $Aloc_t^{vuln}$ which are fit using linear regression.

Type of actions (orange nodes) fit empirically:

$ReduV_T$	Reduce vulnerability action
$RestN_T$	Restore node action
$IsoN_T$	Isolate node action
$ReconN_T$	Reconnect node action

Costs of actions (red nodes, bottom right in Fig. 4), fit with linear regression:

C_T^{ReduV}	Cost for reducing vulnerability
C_T^{RestN}	Cost for restore action
C_T^{IsoN}	Cost for isolate action
C_T^{ReconN}	Cost for reconnect action

Costs due to network status (red nodes, top in Fig. 4), fit with linear regression:

C_T^{iso}	Cost for isolated nodes
C_T^{comp}	Cost for compromised nodes
R_T^{TERM}	Termination cost, when loosing or winning an episode
R_T^{tot}	Total rewards at time T.

The causal inference tasks, in particular, drawing intervention samples from the fitted SCM, are conducted in the Microsoft DoWhy [59] framework. Note that any computational speed implications due to this library are only present during training and consequently do not impact evaluation.

4.6. Libraries and hardware

We note that DoWhy is not optimised for RL training. Its intended use is in the big (not fast) data-regime. A fast, lightweight causal inference package or library for Python would be of high utility for any further investigations.

We use the Meta MBRL-Lib [60] as a harness for this work. For the learning agents, we experimented with both a SAC agent provided by MBRL-Lib, and a PPO agent provided by SB3 [61]. Most of our experimentation and results are conducted with the PPO agent. Training visualisation is provided with TensorBoard, which is part of the TensorFlow [62] package. Our NN models are constructed using PyTorch [63]. On the hardware side, we accelerate training using an NVIDIA 3060Ti/RTX6000 and CUDA [64] where possible.

4.7. Experimental set-up and training phase

We trained three types of agents in the four scenarios as described in Section 4.2. Our research hypothesis in this work is that a causally-aware agent leveraging expert knowledge regarding the cause and effect dynamics shaping the value of a reward would outperform causally-blind agents. Therefore, our experimental set-up compares the proposed causally-aware PPO agent against its conventional model-free and NN model-based versions. Thus, the three distinct agent types are:

- a model-free PPO agent, provided through SB3.
- a model-based PPO agent, equipped with a NN-based transition model T and rewards model R , referred to as NN agent.
- our proposed model-based PPO agent, equipped with a NN-based transition model T and a causally-aware rewards model R , referred to as causal or CAICS agent.

During training, each agent was trained under each scenario multiple rounds, as detailed in Table 1. Specifically, there were 99 and 139 agents trained with standard and HVT reward function, respectively. Each agent was trained for 128K steps, thus, resulting in a total of $(99 + 139) * 128K \approx 30.5$ million steps in the YT environment. We note that, for some general RL use cases, this value may be a small amount of experience for a model-free PPO agent. However, at 128K steps, a convergence is observed in all three agent types, in terms of training loss as seen Fig. 5 and discussed in Section 5. In addition, a convergence to the scenario's max episode length of 500 steps was clearly observed for the NN model-based and causal agents. Thus, 128K steps is assigned for all agents for comparison purposes.

Table 1

Number of agents trained for each repeated experiment across each scenario and reward function. Note that each of the agents listed was trained for 128K steps, with a maximum duration of 500 steps in a game. Thus, a resulting $(99 + 139) * 128K \approx 30.5$ million steps in YT.

Algorithm	Scenario	# of agents for standard reward	# of agents for HVT reward
Our algorithm	High red skill, medium network:	7	11
	High red skill, small network:	8	12
	Low red skill, medium network:	8	13
	Low red skill, small network:	9	13
	Total	32	49
Model-Free	High red skill, medium network:	8	11
	High red skill, small network:	9	10
	Low red skill, medium network:	8	10
	Low red skill, small network:	10	13
	Total	35	44
NN Model-Based	High red skill, medium network:	7	12
	High red skill, small network:	8	09
	Low red skill, medium network:	8	13
	Low red skill, small network:	9	12
	Total	32	46
Overall total		99	139

The Bellman discount factor, gamma, was fixed at 0.99 (the SB3 default), and the rollout buffer sizes were fixed to 256 per environment. Both model-based methods were permitted a very large burn-in phase, of 500K steps, in order to ensure that the model was calibrated, and to ensure that we are comparing the best performing agents in each instance. During training, we monitor:

- The mean, over the last 100 (the default SB3 value) policy updates, for episode length and rewards, which is our main diagnostic for observing ‘learning’ an action correctly (Fig. 5a and b).
- The explained variance between the value function and observed returns (a measure akin to the standard R^2 , of goodness-of-fit) calculates if value function is a good predictor of the returns. Explained variance ranges from $-\infty$ to 1, with 1 indicating that the value function can perfectly predict future returns. This is a measure of the fidelity of the variance of the critic within the PPO actor-critic agent (as illustrated in Fig. 5c).
- Training loss decomposed into policy gradient, value and entropy losses as seen in Fig. 5f, e and d, respectively. PPO is a policy gradient method. Therefore, its total loss calculation includes a loss element, referred to policy gradient loss, based on gradients of expected rewards with respect to policy parameters. Secondly, the value loss described the error between the predicted value function and the actual rewards obtained from the environment. Minimising this type of loss would lead to more accurate predictions of future rewards. Finally, the entropy loss encourages the agent to keep exploring actions by penalising low-entropy policies. Therefore, this behaviour maintains a level of randomness when an agent takes actions, ensuring that the policy does not become too deterministic.

5. Results

In this section we present results for the training and evaluation of our on-policy algorithm 1. All results refer to the use of the standard rewards function, unless explicitly mentioning the HVT rewards function (Section 5.4).

5.1. Training phase metric performance

As seen in Fig. 5a, b, the rollout metrics of mean episode rewards and mean episode length for the causal agent are performing better than either of the other two methods. The learning behaviour for the causally-aware and model-free agents indicates an increasing rewards as it gets trained. The NN model-based has a swift pick on its reward early on but then drops. The proposed CAICS algorithm achieves higher

mean episode rewards than the rest. The behaviour of the mean episode length is similar to the behaviour of the mean episode reward, and the CAICS agent achieves higher mean episode rewards than the rest, indicating a longer longevity throughout the duration of the game.

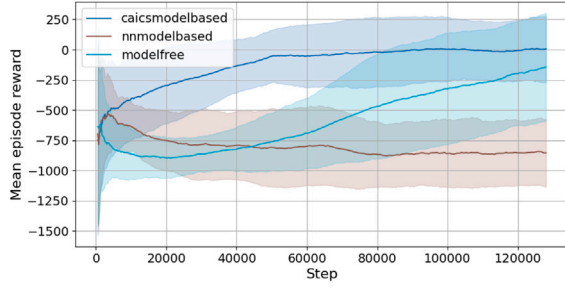
The NN model-based agent performed poorly across these rollout metrics. This may be attributed to the fact that training a single NN on both rewards and dynamics was found to be much harder than training a network to predict the transition dynamics alone. This may also be the reason that there is not much observable advantage to using the neural network model-based agent over the model-free agent. The amount of modelling error with the NN approach is negating the benefits of sample efficiency over the model-free agent. This type of modelling error was something we did not see in the rewards obtained using the SCM, which generally correlate well with YT (see Section 5.5).

We note that within our results it is possible to observe policy collapse. Because this was distributed evenly across agent types, as well as the fact each of the methods is subject to two-of-the-three items in the ‘deadly triad’ (i.e. function approximation and bootstrapping) for RL systems [52], we do not believe the system described in Algorithm 1 is a fundamental cause of this policy collapse behaviour. Instead, the root of this policy collapse is a commonly seen and known issue with RL when applied to large and complicated systems.

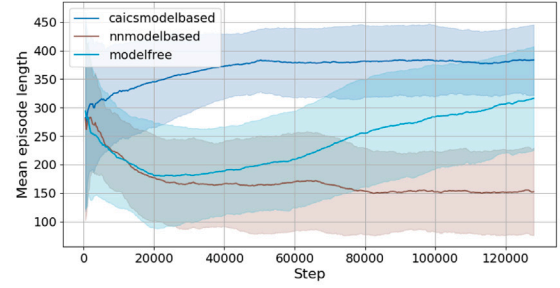
The explained variance (Fig. 5c) for all algorithms is increasing from the onset of the runs. The causally-aware CAICS agent seems to be the most stable one and model-free has a visible increase in its value. However, for the NN model-based algorithm, despite the higher explained variance values, the metric manifests very bursty behaviour, indicating a convergence issue with the agent’s learning.

The change in entropy loss reflects an agent moving from a random policy, with entropy loss equal to the logarithm of the dimension of the action space, i.e. $-\ln d_a = -\ln(72)$ in the case of a small network (18 nodes * 4 actions). This is the entropy of a maximally random policy, moving towards a policy with less entropy. In other words, an entropy loss value approaching zero indicates an agent becoming more certain on which action to deploy. For all types of agents, the entropy loss is decreasing with an increasing number of steps.

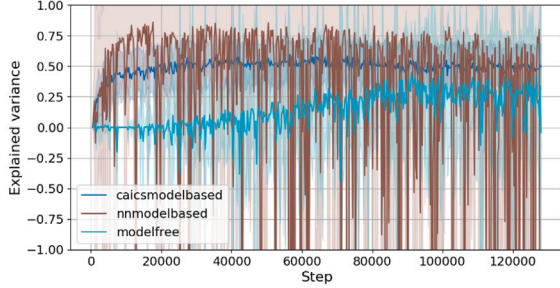
In terms of value and policy gradient loss, all agent types seem to converge early on, with the exception of the value loss for the model-free agent. By comparing these losses between the CAICS algorithm and either the model-free or model-based, it is visible that CAICS manifests slightly higher loss. However, its shaded area, which depicts the standard deviation, shows much smaller variability, indicating a smoother learning behaviour.



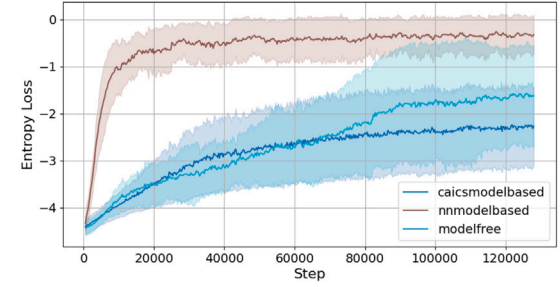
(a) Learning behaviour based on the mean episode reward for each algorithm against the number of steps.



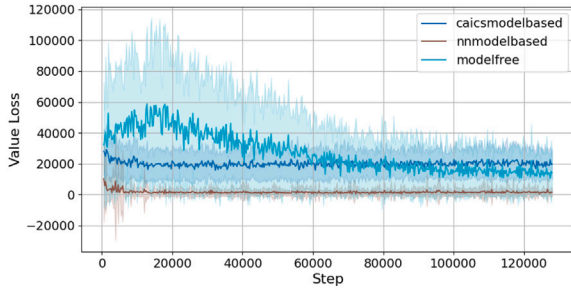
(b) Learning behaviour based on the mean episode length for each algorithm against the number of steps.



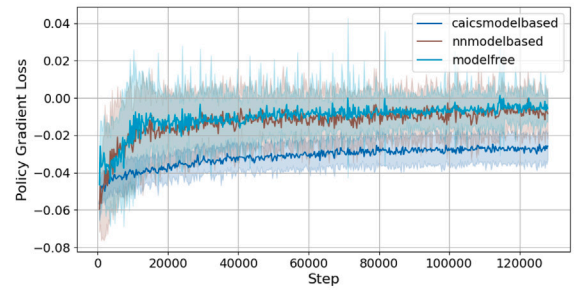
(c) Explained variance for each algorithm against the number of steps. The explained variance for the NN agent has a very bursty behaviour, indicating a convergence issue with its learning behaviour.



(d) An entropy loss value approaching zero indicates an agent becoming more certain on which action to deploy.



(e) Value loss for each algorithm against the number of steps. The value loss is an indication of how well the agent estimates future rewards.



(f) Policy gradient loss for each algorithm against the number of steps. The trend of the loss (decreasing with the number of steps) is where the focus is, rather than the absolute value.

Fig. 5. Six rollout metrics for each of the three algorithms evaluated: model-free, NN model-based and our proposed “CAICS”, causal inference driven algorithm. The graphs are aggregation from multiple runs, as detailed in Section 4.7, with each run encompassing 50 YT simulated games.

5.2. Evaluation phase metrics

We also periodically measure instantaneous scores of additional evaluation metrics for the agent by pausing training to complete an evaluation round. Specifically, during training (Section 5.1), we save the best agent, according to the highest win-rate that each agent achieves. The evaluation was carried out 128 times during training and each evaluation round consisted of 50 YT games. This evaluation cycle is constructed by preparation of a secondary environment, identical but independent to the training environment. We evaluate our agents over 50 games using deterministic policy sampling (i.e. performing the highest probability action rather than randomly sampling from the distribution) in this secondary environment, and record:

- How many games our blue agents were able to ‘win’ against the red agent, referred to as the ‘win-rate’.
- The mean (and standard deviation) episode length over those games.

We found it illustrative to consider how *gracefully* agents interact with YT. Given that agents trained within YT have a preference for using node isolation to win games (causing availability impact), and that the spread of compromise can cause other (integrity or confidentiality) impacts, we found it helpful to also keep record of:

- The percentage and standard deviation of the proportion of the network that is isolated upon a blue agent win.
- The percentage and standard deviation of the proportion of the network which is compromised upon a blue win.

Besides the winning/losing statistics of the game, the above metrics allow us to build a picture of the network’s general health based on isolated and compromised nodes. All of the above metrics are supported by the YT simulation engine and provide context on the overall “health” of the network, besides simple win rate statistics.

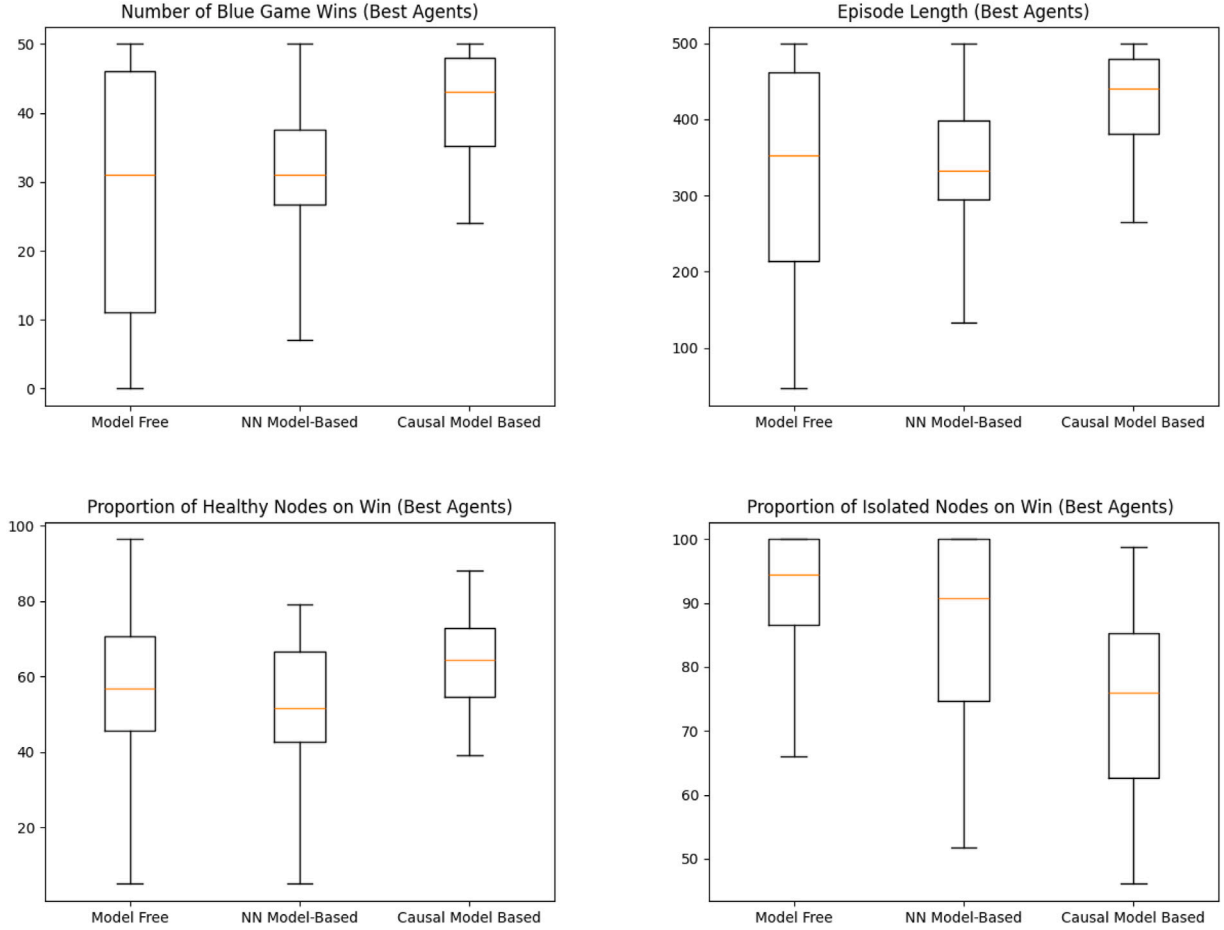


Fig. 6. Box-and-whisker plots detailing best agent (from 50 runs) evaluation performance across all the scenarios using standard rewards function.

Table 2

Median values of each metric for each evaluated algorithm using the standard reward function, as illustrated in Fig. 6. Based on these values, the performance improvement with the standard reward function is calculated in Tables 3 and 4 using Eq. (4).

Metric:	Model free	NN model based	Causal model based
BA wins	31%	31%	43%
Episode length	353.14	333.11	439.71
Healthy nodes on win	56.66%	51.73%	64.35%
Isolated nodes on win	94.5%	90.79%	76%

Table 3

Improvement for our causally-aware algorithm over NN model-based using the standard reward function (top of table) and HVT reward function (bottom of table).

Reward & Scenario	Blue wins	Episode length	Healthy nodes	Isolated nodes
Standard rewards				
Overall	28%	24%	20%	19%
High red skill, medium network	13%	11%	6%	4%
High red skill, small network	39%	37%	12%	9%
Low red skill, medium network	17%	10%	23%	17%
Low red skill, small network	23%	20%	11%	42%
HVT rewards				
Overall	17%	12%	23%	18%
High red skill, medium network	33%	30%	22%	-2%
High red skill, small network	12%	11%	3%	2%
Low red skill, medium network	24%	15%	27%	24%
Low red skill, small network	-11%	-9%	24%	39%

5.3. Agent performance across all scenarios for each evaluated algorithm

Fig. 6 shows box-and-whisker plots corresponding to results of the best evaluation round (128 evaluation rounds, each running 50 YT games as mentioned in Section 5.2) for each of the three agent types, across all four scenarios.

As shown in Fig. 6, the rewards from the proposed causally aware agent outperform the NN model-based agent across the evaluation metrics. That being said, the NN model-based solution does show less overall variance as well as generally less isolated node proclivity than the model-free agent. The causal agent also outperforms the model-free agent, which was to be expected due to the sample-restricted training.

Across all scenarios the causal agent obtains a higher win-rate during evaluation. Furthermore, when the agent does win YT, there are more healthy nodes and a lower number of isolated nodes compared

to the benchmarks. Table 2 shows the median values of all evaluation metrics, as illustrated in Fig. 6, for each algorithm across all experiment runs using the standard reward function. Based on these values, the performance improvement, as shown in Table 3, is calculated by:

Performance improvement

$$= \frac{\text{Causal median value} - \text{NN Model Based value}}{\text{Causal median value}} * 100\% \quad (4)$$

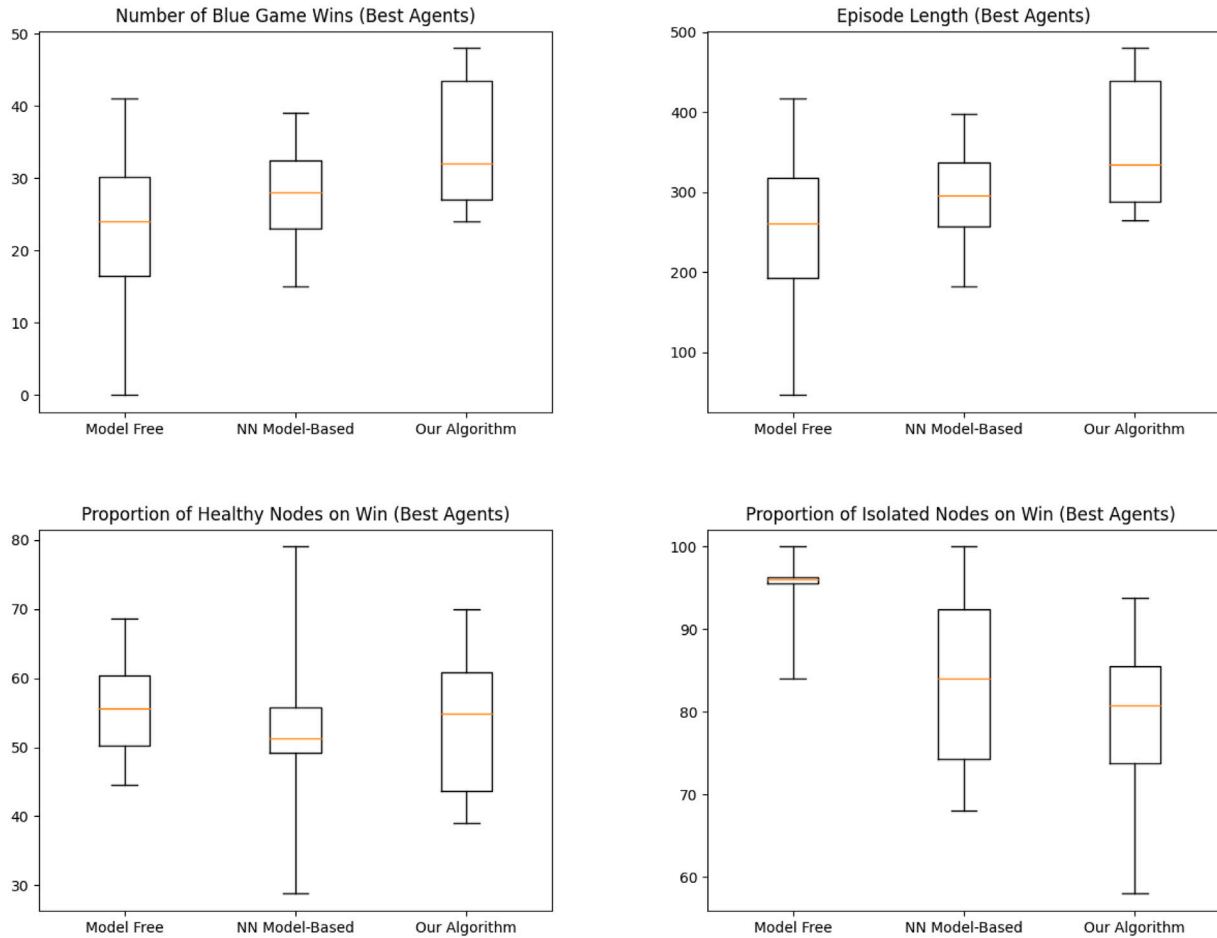


Fig. 7. Box-and-whisker plots detailing best agent (from 50 runs) evaluation performance for the “high red skill, medium network” scenario using standard rewards function.

Table 4

Improvement for our causally-aware algorithm over model-free using the standard reward function (top of table) and HVT reward function (bottom of table).

Reward & Scenario	Blue wins	Episode length	Healthy nodes	Isolated nodes
Standard rewards				
Overall	28%	20%	12%	24%
High red skill, medium network	25%	22%	−2%	19%
High red skill, small network	−1%	−1%	−8%	23%
Low red skill, medium network	68%	42%	25%	7%
Low red skill, small network	11%	7%	15%	52%
HVT rewards				
Overall	40%	29%	19%	17%
High red skill, medium network	68%	62%	17%	7%
High red skill, small network	0%	0%	4%	2%
Low red skill, medium network	30%	19%	28%	24%
Low red skill, small network	38%	29%	3%	53%

and similarly for the model-free agent, as shown in Table 4. For brevity we do not include a Table with the individual median values for the HVT reward function experiments, but rather only present the percentage performance increase at the second half of Tables 3 and 4.

Quantitative results for each individual scenario comparing the performance of our causal agents against the NN and model-free can be found in Tables 3 and 4, respectively. Overall performance, in terms

of blue game wins, exceeded that of both model-free and NN model-based agents by 28% (calculated from the median values in Table 2: $(43\% - 31\%)/31\%$). This increase in performance for the causal agent is also manifested in terms of episode length, indicating the longevity of the agent. Specifically, the episode length is higher by 20% and 24% in comparison to the model-free and NN model-based agents, respectively. In terms of number of healthy nodes at the end of the game, the results improved by 12% and 20% against the model free and NN agents, respectively. Finally, in terms of number of isolated nodes at the end of the game, our causal agent has a 24% improvement against model free and 19% against model-based agents.

Box-and-whisker plots for each individual scenario are shown in Figs. 7–10. As seen in the figures, for nearly all scenarios, the proposed causally-aware approach has better median performance for all four metrics. The only exception is the “High red skill, small network” scenario, where the model-free agent outperforms the causal and NN agents in terms of healthy nodes metrics. In this scenario, the model-free is also on par in performance in comparison to the causal agent in terms of blue wins and episode length.

From the individual scenario results (Tables 3 and 4), we note that the causally-aware blue agent achieves large percentage improvements against the NN model-based agent, with respect to Blue Wins and Episode Length, in the small network scenarios. On the other hand, larger performance improvements for the metrics of Healthy and Isolated Nodes is occurring against a low red skill agent, independently of the network size.

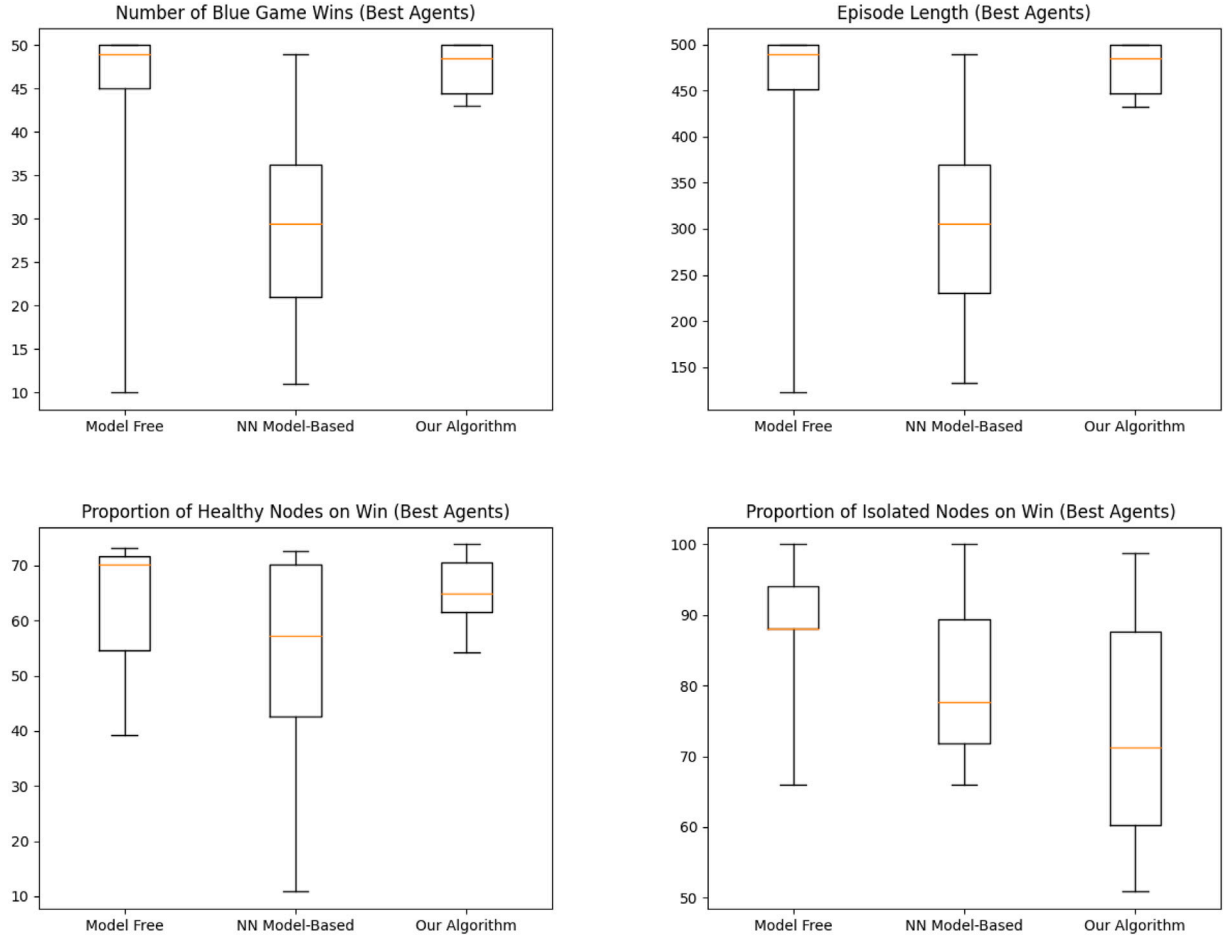


Fig. 8. Box-and-whisker plots detailing best agent (from 50 runs) evaluation performance for the “high red skill, small network” scenario using standard rewards function.

In contrast, when comparing the causally-aware agent against the model-free agent, the same conclusions as in the NN case do not hold. In particular, the bottom half of Table 4 indicates large percentage improvement for the medium size network. However, it still holds that the Healthy and Isolated Nodes present large percentage improvements against a low red skill agent. Finally, the only non-significant (−8%) drop in performance for the causally-aware agent happens against the model-free agent, during the “High red skill, small network scenario”, for the Healthy Nodes metric.

5.4. HVT reward function

As observed in Fig. 11, the HVT rewards function modification did not result in any noticeable degradation, with respect to the performance of the agent. Specifically, in terms of blue wins and episode length, compared to the standard reward function results (see Fig. 6), the HVT reward function resulted in an improvement for the NN model-based agent and, to a lesser extent, for the causally-aware agent. Note that the performance of the proposed causally-aware agent with the standard reward function was already very high, so did not show such large improvements as the NN model-based agent.

Again, in comparison to the standard rewards function, beyond the metrics of blue agent wins and episode length, the HVT reward function also led to an improvement in the proportion of healthy nodes, but at a small increase in the percentage of isolated nodes.

Quantitative results for each scenario comparing the performance of our proposed agents against the model-free and NN, using the HVT reward function can be found at the bottom half of Tables 3 and 4, respectively. In terms of “blue wins”, the overall performance metric exceeded that of model-free and NN model-based agents by 40% and 17%, respectively.

In terms of number of healthy nodes at the end of the game, the results improved by 19% and 23% against the model free and NN agents, respectively. In terms of number of isolated nodes at the end of the game, the HVT reward function leads to 17% and 18% improvement against the model free and NN model-based approach. Note, that the number of isolated nodes may include any type of node, not just HVT nodes. Note that the proposed HVT reward function achieves these benefits in comparison to the standard reward function even though it strongly discourages applying disrupted actions on HVT nodes. This is an important distinction as it is frequently observed for the standard reward function in YT to lead to disconnected HVT nodes. The only non-significant drop in performance for the HVT reward function is observed against the NN model-based agent in the “Low red skill, small network” scenario. Specifically the blue wins and episode length drop by −11% and −9%, respectively.

Finally, another difference with the HVT reward function is in the action distribution results for each agent. The agents trained with the HVT reward function indicated, in some cases, a more balanced action distribution during evaluation.

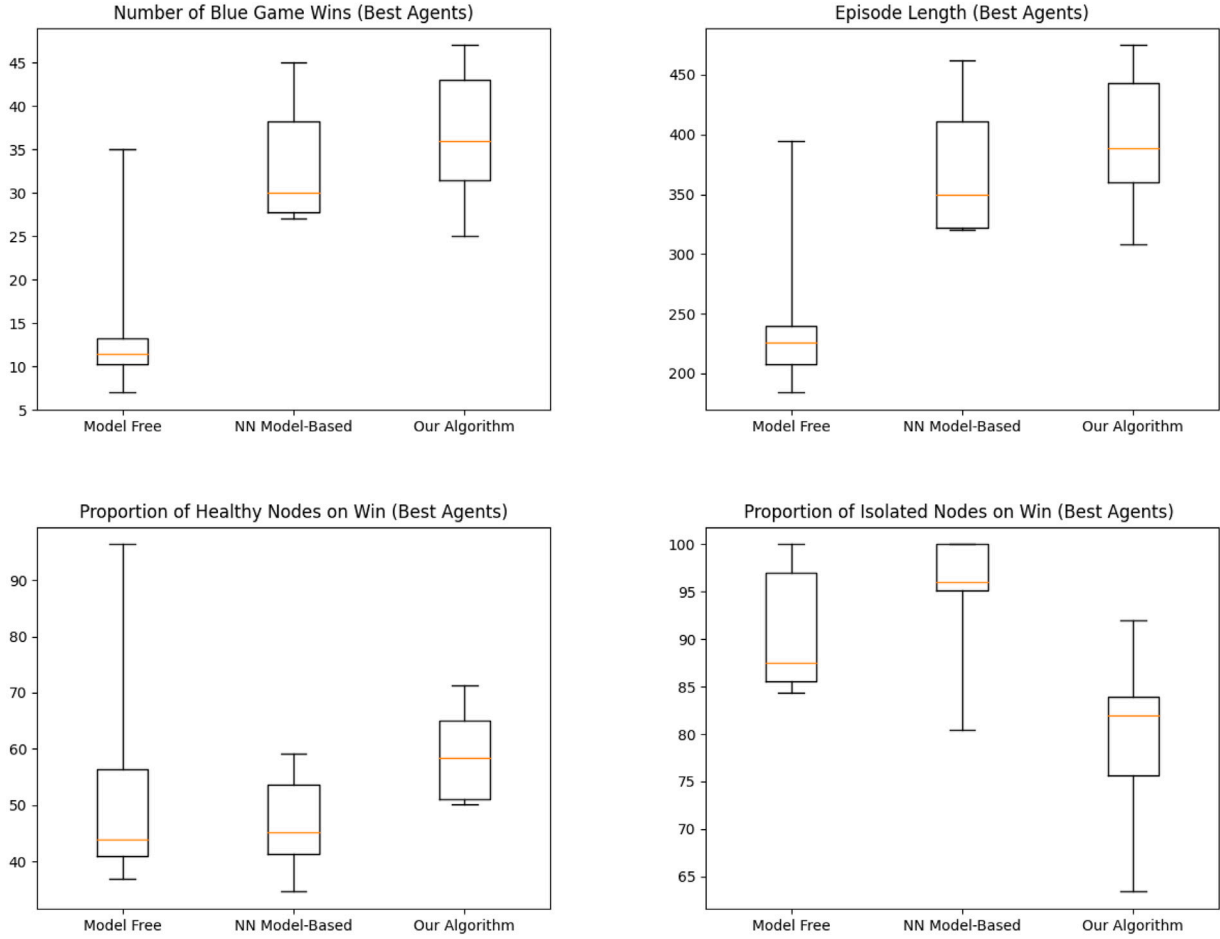


Fig. 9. Box-and-whisker plots detailing best agent (from 50 runs) evaluation performance for the “low red skill, medium network” scenario using standard rewards function.

5.5. Causal rewards fidelity comparison

In order to check the fidelity of our causal rewards model against YT, we log the SCM-generated and YT-sampled reward values for a single training run of Algorithm 1 from the ‘high red skill, small network’ scenario. Fig. 12 is the result of this recording. We note that despite YT having a complex rewards structure, there is good agreement between the shape of this and the SCM rewards distribution, including both ‘tails’ of the reward spectrum (although, the losing condition appears to be calibrated a few units away from its true value). This indicates that the SCM is able to accurately mimic both the rewards for winning and losing, as well as the complex structure of the rewards issued during an episode.

Similarly, the same approach is followed to compare the NN-generated rewards against the YT-sampled reward values from the ‘high red skill, small network’ scenario. However, as seen in Fig. 13, the NN sample rewards do not represent the rewards from the YT environment as accurately as the SCM model. Note, the count of NN samples is much larger because rewards from all 8 simulated environments (as described in Section 4.1) have been aggregated into the histogram, thus resulting into much higher reward counts than the ones from YT. However, the important point is to focus on the distribution behaviour. Specifically, the YT reward count peaks around -1.4 and values are mostly within the $[-1.5, -0.25]$ range. In contrast, the NN reward count peaks around 0.01 . Furthermore, NN fails to mimic both rewards for winning and

losing states as illustrated in the ‘tails’ of the rewards spectrum (top and bottom inset plots in Fig. 13), where even though we have YT samples, there are no NN samples.

6. Conclusions

The overarching aim of this work is to improve the performance of RL agents when autonomously defending networks against cyber attacks. Through this work we have successfully integrated causal modelling together with RL algorithms and the YT environment. This investigation integrated multiple software frameworks, including MBRL-Lib, DoWhy and SB3.

We have demonstrated how causal modelling can aid in creation of trained RL agents by reducing the required interactions with the training environment needed to achieve optimal performance. This, to our knowledge, is the first work of its kind. Our causal model is serving the role of a rewards model. This permits the agent, *in its model*, to represent the cause-and-effect of its own actions and learn from that experience.

Our approach is not limited to the scope of cyber-security, since our causal algorithm is sufficiently generic. To apply our approach to a particular problem, one only needs to model the cause and effect SCM for that specific environment. This approach can therefore be applied to any RL challenge where environmental rewards can be modelled.

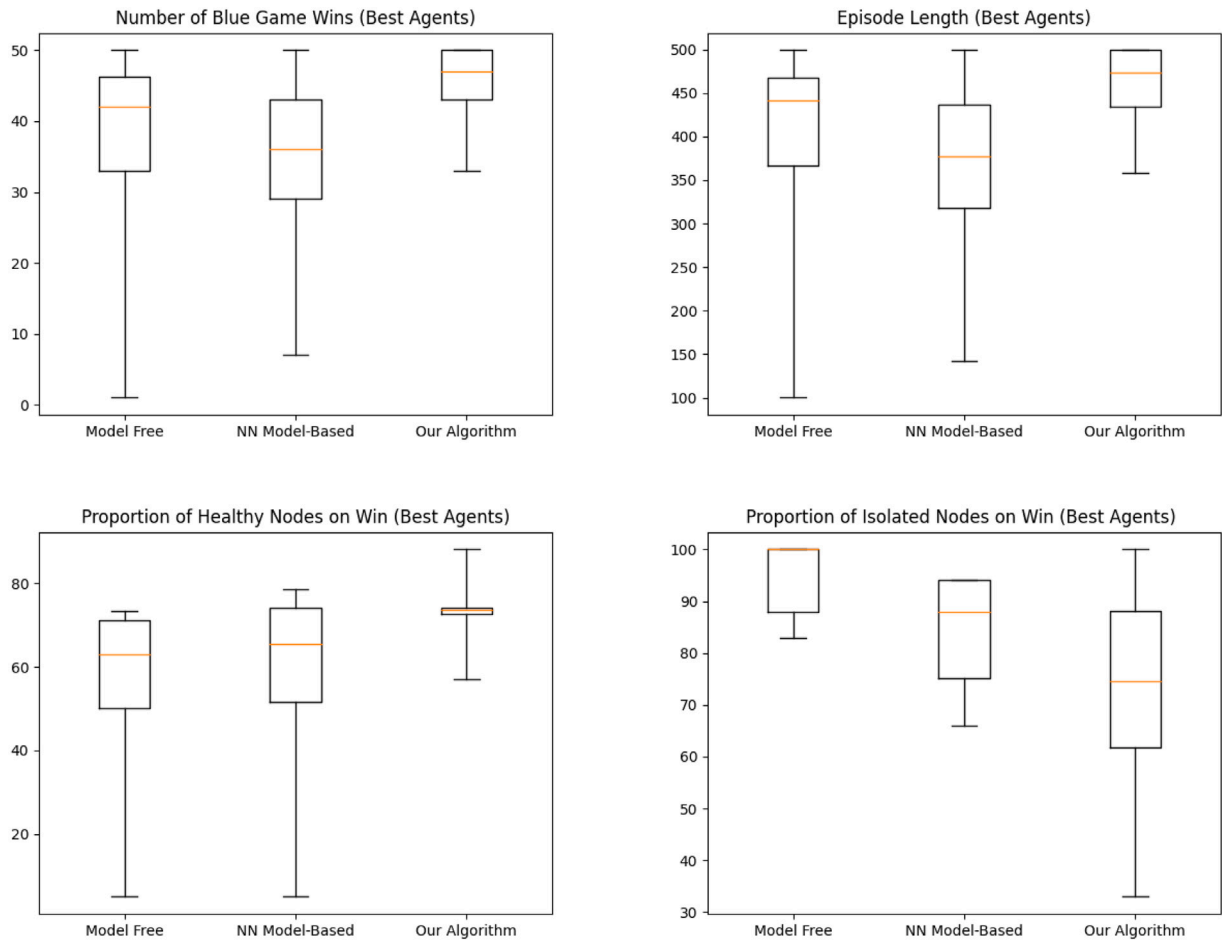


Fig. 10. Box-and-whisker plots detailing best agent (from 50 runs) evaluation performance for the “low red skill, small network” scenario using standard rewards function.

For Autonomous Resilient Cyber Defence (ARCD) [65], our approach is compatible with the training environments developed by Dstl, such as YT [40] or PrimaITE [43]. Our algorithmic approach could be applied as an extension to any standard model-free agent in the same way that was demonstrated with SB3 PPO in this work.

We have applied causal inference to an adversarial cyber security problem and have shown that it is robust across four distinct scenarios, ensuring our methods are robust against different red agent skill levels and network sizes. We have optimised an RL agent using a variety of methods, and we have reported on the improved performance of our agent against causally-blind benchmarks through the use of relevant metrics and visualisations.

Specifically, we have investigated the performance of our agent through the gamified ‘win-rate’ metric, as well as looking into a variety of ancillary training metrics for gaining insight on how, RL agents play, and win the game of YT. We have also investigated an alternative reward function to discourage the agent from applying mitigation actions on the HVT node, which would limit impact on the CIA requirements and the network’s mission function.

Overall, for the proposed HVT rewards function, the performance of our approach, in terms of blue game wins, exceeded that of model-free and NN model-based agents by 40% and 17%, respectively. In addition, the overall “healthy nodes” performance improvement was 19% and 23% against the model-free and NN model-based agents, respectively.

Finally, the overall “isolated nodes” metric was improved by 17% and 18%, against the model-free and NN model-based agents, respectively.

7. Future work

Our work has led to several exciting research questions that we aim to address in future work. For example, how might we design multi-agent frameworks for causal inference in cyber-battlespace environments that form a consistent picture across and between agents? Even though the mathematical foundations for such work have been partially addressed in the literature [66], decentralising control is still very much at the research boundary. The problem is complicated further by any notions of partial observability, which we do not tackle at all in this current work. There is scope to extend the investigation towards an adversarial multi-agent game, as well as incorporating a training process based on self-play [67].

There are potential methods to extend our proposed approach. One advantage of the causal rewards model (over a NN rewards model tacked-on to the dynamics model) is that it is interpretable. The great thing about having interpretable models is that our understanding of this model can be expanded, perhaps even outside of the observation space of the RL agent.

Furthermore, the concept of counterfactuals from CI marries nicely with the related RL concept of regret [68]. Regret, semantically and

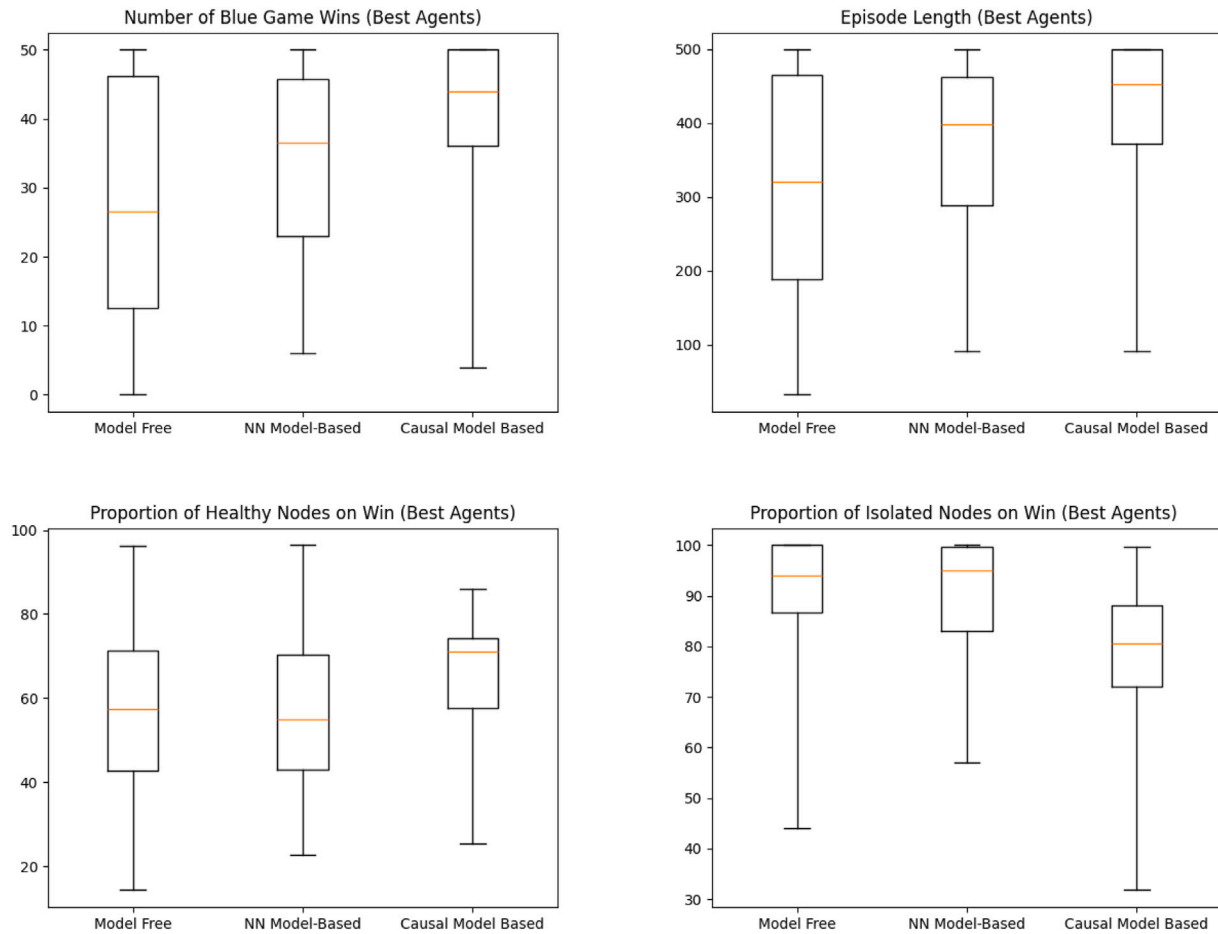


Fig. 11. Box-and-whisker plots using the HVT reward function detailing best agent evaluation performance across all scenarios.

necessarily, involves a counterfactual (colloquially, a ‘what-if’ statement). DRL systems have been shown to be highly vulnerable to adversarial noise-based attack. Regret-based learning [69] is one of the approaches that could ameliorate this issue in a proactive manner by improving robustness. Therefore, using counterfactuals to provide resilience to DRL against adversarial approaches would be an interesting approach.

Finally, as long as the agent is receiving *some* information from outside its ‘environment’, the model could augment the intrinsic environment reward according to extrinsic factors. Applied to the cyber domain, this might allow the development of agents that only operate on localities of a network but receive additional information about global network state. Or this could be more mission driven: agents might be trained to respond to different threats by taking into account information about the attack using CIA impact.

CRedit authorship contribution statement

Tom Purves: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Konstantinos G. Kyriakopoulos:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software,

Resources, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Siân Jenkins:** Visualization, Validation, Software, Investigation, Data curation. **Iain Phillips:** Writing – review & editing, Supervision, Software, Resources, Investigation. **Tim Dudman:** Writing – review & editing, Supervision, Software, Resources, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgements

Research funded by Frazer-Nash Consultancy Ltd. on behalf of the Defence Science and Technology Laboratory (Dstl) which is an executive agency of the UK Ministry of Defence providing world class expertise and delivering cutting-edge science and technology for the benefit of the nation and allies. The research supports the Autonomous Resilient Cyber Defence (ARCD) project within the Dstl Cyber Defence Enhancement programme.

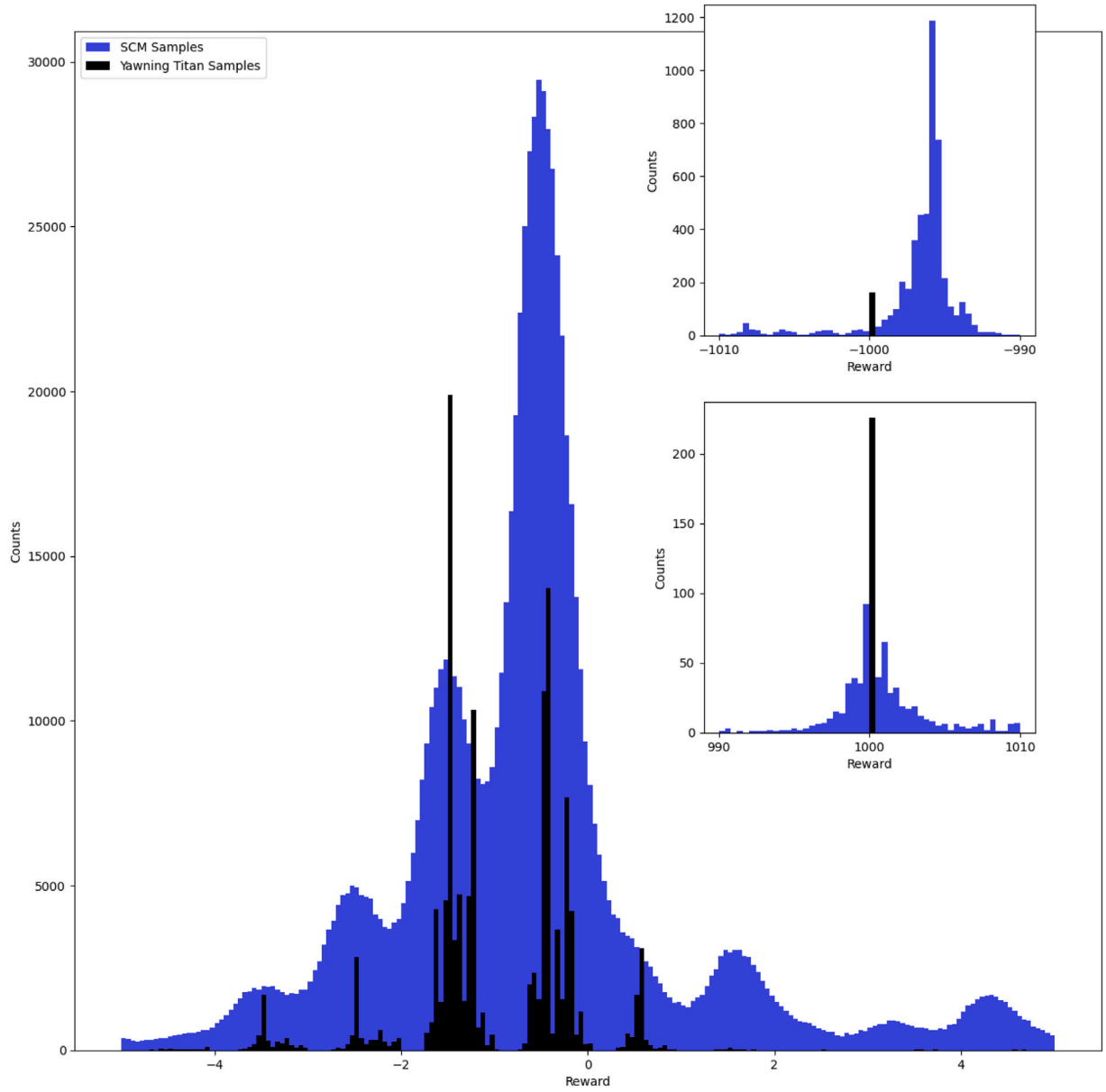


Fig. 12. Comparison of rewards generated with the YT environment and utilising interventions on the SCM, as illustrated in Fig. 4. The inset plots show the edges $[-1010$ and $1010]$ of the x-axis.

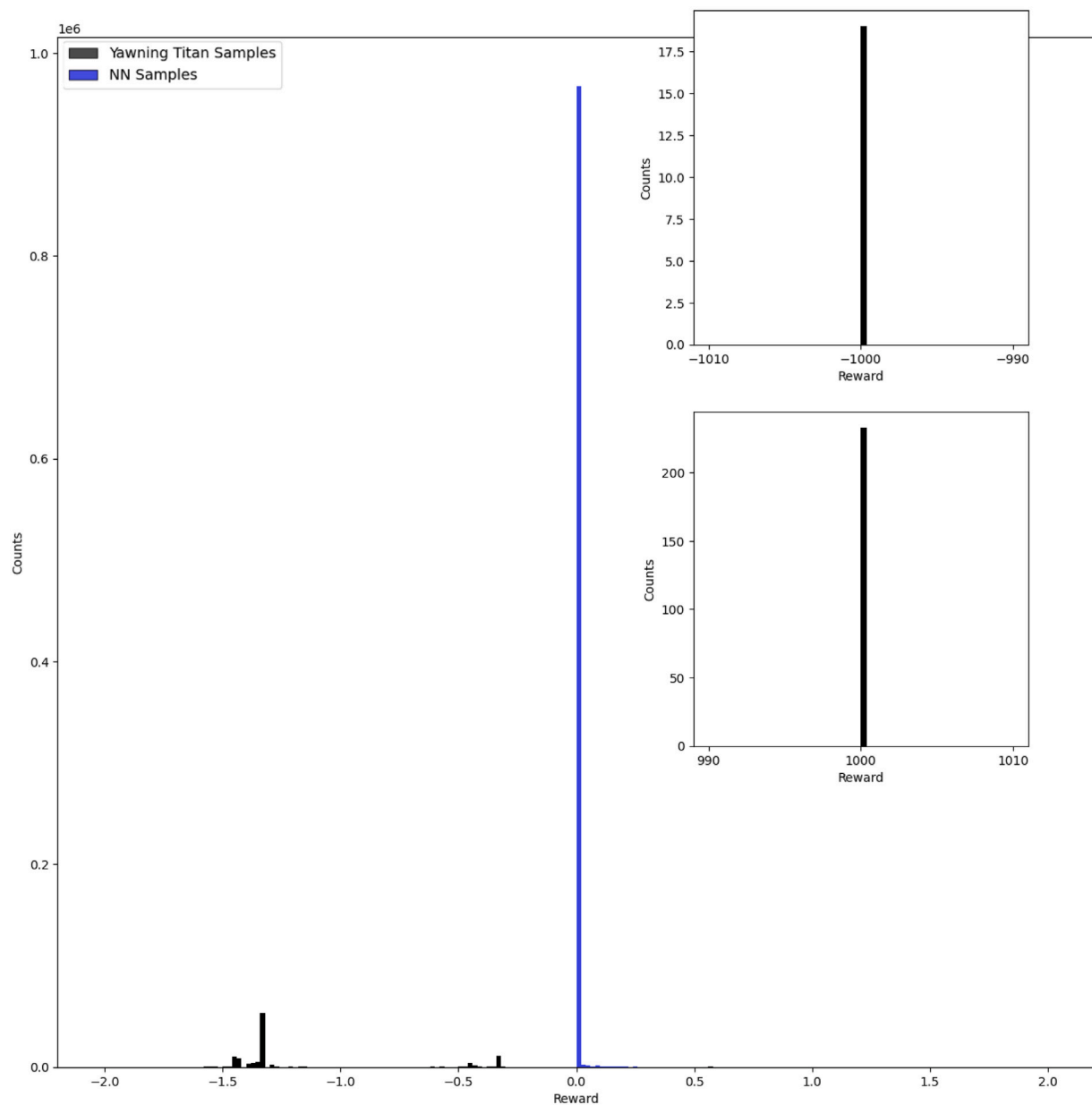


Fig. 13. Comparison of rewards generated with the YT environment and the NN model-based approach. The inset plots show the edges around -1000 (agent losing) and 1000 (agent winning).

References

- [1] Sophos whitepaper, The impossible puzzle of cybersecurity, 2019, URL <https://www.sophos.com/en-us/whitepaper/theimpossible-puzzle-of-cybersecurity>, Available at <https://www.sophos.com/en-us/whitepaper/theimpossible-puzzle-of-cybersecurity>.
- [2] N. Dhir, H. Hoeltgebaum, N. Adams, M. Briers, A. Burke, P. Jones, Prospective artificial intelligence approaches for active cyber defence, 2021, arXiv preprint [arXiv:2104.09981](https://arxiv.org/abs/2104.09981).
- [3] J. Biden, K. Harris, National cybersecurity strategy, 2023, URL <https://www.whitehouse.gov/wp-content/uploads/2023/03/National-Cybersecurity-Strategy-2023.pdf>.
- [4] L. Fridman, Transcript for Benjamin Netanyahu: Israel, Palestine, Power, Corruption, Hate, and Peace, 2023, URL <https://lexfridman.com/benjamin-netanyahu-transcript/>.
- [5] T.T. Nguyen, V.J. Reddi, Deep reinforcement learning for cyber security, *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [6] W. Zhao, J.P. Queralta, T. Westerlund, Sim-to-real transfer in deep reinforcement learning for robotics: a survey, in: 2020 IEEE Symposium Series on Computational Intelligence, SSCI, IEEE, 2020, pp. 737–744.
- [7] F. Berkenkamp, M. Turchetta, A. Schoellig, A. Krause, Safe model-based reinforcement learning with stability guarantees, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [8] J. Zhang, B. Cheung, C. Finn, S. Levine, D. Jayaraman, Cautious adaptation for reinforcement learning in safety-critical settings, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 11055–11065.
- [9] Q. Liu, A. Chung, C. Szepesvári, C. Jin, When is partially observable reinforcement learning not scary? in: *Conference on Learning Theory*, PMLR, 2022, pp. 5175–5220.
- [10] R. Noothigattu, D. Bouneffouf, N. Mattei, R. Chandra, P. Madan, K.R. Varshney, M. Campbell, M. Singh, F. Rossi, Teaching AI agents ethical values using reinforcement learning and policy orchestration, *IBM J. Res. Dev.* 63 (4/5) (2019) 2–1.
- [11] M. Deisenroth, C.E. Rasmussen, PILCO: A model-based and data-efficient approach to policy search, in: *Proceedings of the 28th International Conference on Machine Learning*, ICML-11, 2011, pp. 465–472.
- [12] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, D. Hassabis, Mastering the game of go without human knowledge, *Nature* 550 (7676) (2017) 354–359, <http://dx.doi.org/10.1038/nature24270>.

- [13] L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R.H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, et al., Model-based reinforcement learning for atari, 2019, arXiv preprint [arXiv:1903.00374](https://arxiv.org/abs/1903.00374).
- [14] J. Pearl, M. Glymour, N.P. Jewell, *Causal Inference in Statistics: A Primer*, John Wiley & Sons, 2016.
- [15] X. Wang, Y. Du, S. Zhu, L. Ke, Z. Chen, J. Hao, J. Wang, Ordering-based causal discovery with reinforcement learning, 2021, arXiv preprint [arXiv:2105.06631](https://arxiv.org/abs/2105.06631).
- [16] L.G. Neuberger, *Causality: models, reasoning, and inference*, by Judea Pearl, Cambridge University Press, 2000, *Econometric Theory* 19 (4) (2003) 675–685.
- [17] C.N. Glymour, *The Mind's Arrows: Bayes Nets and Graphical Causal Models in Psychology*, MIT Press, 2001.
- [18] S. Sloman, *Causal Models: How People Think About the World and Its Alternatives*, Oxford University Press, 2005.
- [19] A.R. Sharma, P. Kaushik, Literature survey of statistical, deep and reinforcement learning in natural language processing, in: 2017 International Conference on Computing, Communication and Automation, ICCCA, IEEE, 2017, pp. 350–354.
- [20] W.Y. Wang, J. Li, X. He, Deep reinforcement learning for NLP, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts, 2018, pp. 19–21.
- [21] V. Uc-Cetina, N. Navarro-Guerrero, A. Martin-Gonzalez, C. Weber, S. Wermter, Survey on reinforcement learning for language processing, *Artif. Intell. Rev.* 56 (2) (2023) 1543–1575.
- [22] L.-J. Lin, Programming robots using reinforcement learning and teaching, in: Proceedings of the Ninth National Conference on Artificial Intelligence-Volume 2, 1991, pp. 781–786.
- [23] J. Kober, J.A. Bagnell, J. Peters, Reinforcement learning in robotics: A survey, *Int. J. Robot. Res.* 32 (11) (2013) 1238–1274.
- [24] P. Kormushev, S. Calinon, D.G. Caldwell, Reinforcement learning in robotics: Applications and real-world challenges, *Robotics* 2 (3) (2013) 122–148.
- [25] M.M. Afsar, T. Crump, B. Far, Reinforcement learning based recommender systems: A survey, *ACM Comput. Surv.* 55 (7) (2022) 1–38.
- [26] Y. Lin, Y. Liu, F. Lin, L. Zou, P. Wu, W. Zeng, H. Chen, C. Miao, A survey on reinforcement learning for recommender systems, *IEEE Trans. Neural Netw. Learn. Syst.* (2023) 1–21, <https://doi.org/10.1109/tnnls.2023.3280161>.
- [27] M. Foley, C. Hicks, K. Highnam, V. Mavroudis, Autonomous network defence using reinforcement learning, in: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, in: ASIA CCS '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1252–1254, <https://doi.org/10.1145/3488932.3527286>.
- [28] M. Foley, M. Wang, Z. M., C. Hicks, V. Mavroudis, Inroads into autonomous network defence using explained reinforcement learning, 2023, [arXiv:2306.09318](https://arxiv.org/abs/2306.09318).
- [29] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, et al., Model-based reinforcement learning: A survey, *Found. Trends Mach. Learn.* 16 (1) (2023) 1–118.
- [30] F.-M. Luo, T. Xu, H. Lai, X.-H. Chen, W. Zhang, Y. Yu, A survey on model-based reinforcement learning, 2022, [arXiv:2206.09328](https://arxiv.org/abs/2206.09328).
- [31] A.S. Polydoros, L. Nalpanidis, Survey of model-based reinforcement learning: Applications on robotics, *J. Intell. Robot. Syst.* 86 (2) (2017) 153–173.
- [32] J. Martinez Torres, C. Iglesias Comesaña, P.J. Garcia-Nieto, Machine learning techniques applied to cybersecurity, *Int. J. Mach. Learn. Cybern.* 10 (2019) 2823–2836.
- [33] Y. Han, B.I. Rubinstein, T. Abraham, T. Alpcan, O. De Vel, S. Erfani, D. Hubczenko, C. Leckie, P. Montague, Reinforcement learning for autonomous defence in software-defined networking, in: Decision and Game Theory for Security: 9th International Conference, GameSec 2018, Seattle, WA, USA, October 29–31, 2018, Proceedings 9, Springer, 2018, pp. 145–165.
- [34] H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, 2015, [arXiv:1509.06461](https://arxiv.org/abs/1509.06461). URL <https://arxiv.org/abs/1509.06461>.
- [35] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: International Conference on Machine Learning, PMLR, 2016, pp. 1928–1937.
- [36] M. Beechey, S. Lambbotharan, K.G. Kyriakopoulos, Evidential classification for defending against adversarial attacks on network traffic, *Inf. Fusion* 92 (2023) 115–126.
- [37] D. Pathak, P. Agrawal, A.A. Efros, T. Darrell, Curiosity-driven exploration by self-supervised prediction, in: International Conference on Machine Learning, PMLR, 2017, pp. 2778–2787.
- [38] CAGE, CAGE challenge 1, in: IJCAI-21 1st International Workshop on Adaptive Cyber Defense., arXiv, 2021.
- [39] M. Standen, M. Lucas, D. Bowman, T. Richer, J. Kim, D. Marriott, CyBORG: A gym for the development of autonomous cyber agents, 2021, [arXiv:2108.09118](https://arxiv.org/abs/2108.09118).
- [40] Defence Science and Technology Laboratory UK, YAWNING TITAN: An abstract, graph based cyber-security simulation environment for autonomous cyber operations., 2023, URL <https://github.com/dstl/YAWNING-TITAN>.
- [41] V. Aglietti, N. Dhir, J. González, T. Damoulas, Dynamic causal Bayesian optimization, 2021, [arXiv:2110.13891](https://arxiv.org/abs/2110.13891).
- [42] A. Andrew, S. Spillard, J. Collyer, N. Dhir, Developing optimal causal cyber-defence agents via cyber security simulation, in: Workshop on Machine Learning for Cybersecurity (MLCyber), 2022.
- [43] Defence Science and Technology Laboratory UK, Primary-level AI training environment (PrimAITE), 2023, URL <https://github.com/Autonomous-Resilient-Cyber-Defence/PrimAITE>.
- [44] L. Li, R. Fayad, A. Taylor, Cygil: A cyber gym for training autonomous agents over emulated network systems, 2021, [arXiv:2109.03331](https://arxiv.org/abs/2109.03331).
- [45] A. Molina-Markham, C. Minitier, B. Powell, A. Ridley, Network environment design for autonomous cyberdefense, 2021, [arXiv:2103.07583](https://arxiv.org/abs/2103.07583).
- [46] Microsoft Defender Research Team, CyberBattleSim, 2021, URL <https://www.microsoft.com/en-us/research/project/cyberbattlesim/>, Available at <https://www.microsoft.com/en-us/research/project/cyberbattlesim/>.
- [47] M. Rigaki, O. Lukáš, C.A. Catania, S. Garcia, Out of the cage: How stochastic parrots win in cyber security environments, 2023, [arXiv:2308.12086](https://arxiv.org/abs/2308.12086).
- [48] M. Towers, J.K. Terry, A. Kwiatkowski, J.U. Balis, G.d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J.J. Tai, A.T.J. Shen, O.G. Younis, Gymnasium, 2023, <https://doi.org/10.5281/zenodo.8127026>, URL <https://zenodo.org/record/8127026>.
- [49] S. Vyas, J. Hannay, A. Bolton, P.P. Burnap, Automated cyber defence: A review, 2023, [arXiv:2303.04926](https://arxiv.org/abs/2303.04926).
- [50] J. Nyberg, P. Johnson, Training automated defense strategies using graph-based cyber attack simulations, 2023, [arXiv:2304.11084](https://arxiv.org/abs/2304.11084).
- [51] J. Collyer, A. Andrew, D. Hodges, ACD-G: Enhancing Autonomous Cyber Defense Agent Generalization Through Graph Embedded Network Representation, International Conference on Machine Learning, 2022.
- [52] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [53] J.W. Forrester, Counterintuitive behavior of social systems, *Theory Decis.* 2 (2) (1971) 109–140.
- [54] M.A. Hernán, J.M. Robins, *Causal inference*, 2010.
- [55] R.S. Sutton, Dyna, an integrated architecture for learning, planning, and reacting, *SIGART Bull.* 2 (4) (1991) 160–163, <https://doi.org/10.1145/122344.122377>.
- [56] M. Janner, J. Fu, M. Zhang, S. Levine, When to trust your model: Model-based policy optimization, 2021, [arXiv:1906.08253](https://arxiv.org/abs/1906.08253).
- [57] P. Manadhata, J.M. Wing, Measuring a System's Attack Surface, School of Computer Science, Carnegie Mellon University Pittsburgh, PA, USA, 2004.
- [58] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perot, E. Duchesnay, Scikit-learn: Machine Learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [59] A. Sharma, E. Kiciman, et al., DoWhy: A python package for causal inference, 2019, <https://github.com/microsoft/dowhy>.
- [60] L. Pineda, B. Amos, A. Zhang, N.O. Lambert, R. Calandra, MBRL-lib: A modular library for model-based reinforcement learning, 2021, [arXiv:2104.10159](https://arxiv.org/abs/2104.10159). URL <https://arxiv.org/abs/2104.10159>.
- [61] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-Baselines3: Reliable reinforcement learning implementations, *J. Mach. Learn. Res.* 22 (268) (2021) 1–8, URL <http://jmlr.org/papers/v22/20-1364.html>.
- [62] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, L. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, TensorFlow: Large-scale machine learning on heterogeneous systems, 2015, URL <https://www.tensorflow.org/>, Software available from tensorflow.org.
- [63] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, A. Lerer, Automatic differentiation in pytorch, in: NIPS-W, 2017.
- [64] D. Luebke, CUDA: Scalable parallel programming for high-performance scientific computing, in: 2008 5th IEEE International Symposium on Biomedical Imaging: From Nano To Macro, 2008, pp. 836–838, <https://doi.org/10.1109/ISBI.2008.4541126>.
- [65] Frazer-Nash Consultancy, Autonomous resilient cyber defence, 2024, URL <https://www.fnc.co.uk/arcd/>, Available on January 2024 at <https://www.fnc.co.uk/arcd/>.
- [66] S. Maes, S. Meganck, B. Manderick, Inference in multi-agent causal models, *Internat. J. Approx. Reason.* 46 (2) (2007) 274–299, <https://doi.org/10.1016/j.ijar.2006.09.005>, URL <https://www.sciencedirect.com/science/article/pii/S088613X06001253>, Special Track on Uncertain Reasoning of the 18th International Florida Artificial Intelligence Research Symposium (FLAIRS 2005).
- [67] A. DiGiovanni, E.C. Zell, Survey of self-play in reinforcement learning, 2021, [arXiv:2107.02850](https://arxiv.org/abs/2107.02850).
- [68] R. Belaire, P. Varakantham, D. Lo, Regret-based optimization for robust reinforcement learning, 2023, [arXiv:2302.06912](https://arxiv.org/abs/2302.06912).
- [69] R. Belaire, P. Varakantham, D. Lo, Robust deep reinforcement learning through regret neighborhoods, 2023, [Arxiv](https://arxiv.org/abs/2302.06912).